

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Nguyễn Thành Phát

Suy luận nhân quả trên dữ liệu chuỗi thời gian

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



NGUYỄN THÀNH PHÁT

SUY LUẬN NHÂN QUẢ TRÊN DỮ LIỆU CHUỖI THỜI
GIAN

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

Cán bộ hướng dẫn: TS. Nguyễn Văn Sơn

HÀ NỘI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Nguyen Thanh Phat

CAUSAL INFERENCE ON TIME SERIES DATA

BACHELOR'S THESIS

Major: Information Technology

Supervisor: Dr. Nguyen Van Son

HA NOI - 2026

Lời cảm ơn

Lời đầu tiên, tôi xin gửi lời cảm ơn chân thành và lòng biết ơn sâu sắc tới thầy TS. Nguyễn Văn Sơn, người đã tận tình hướng dẫn tôi xuyên suốt thời gian thực hiện khóa luận tốt nghiệp này.

Tôi xin cảm ơn đội ngũ CrunchDAO nói chung và anh Enzo Caceres nói riêng đã tạo ra một nền tảng, sân chơi và hỗ trợ tôi xuyên suốt quá trình thực hiện khóa luận tốt nghiệp này.

Tôi cũng xin gửi lời cảm ơn chân thành tới các thầy, cô hiện đang công tác và giảng dạy tại trường Đại học Công Nghệ - Đại học Quốc gia Hà Nội đã tạo mọi điều kiện thuận lợi cho tôi học tập và nghiên cứu.

Hà Nội, ngày tháng năm

Sinh viên

Nguyễn Thành
Phát

Lời cam đoan

Tôi là Nguyễn Thành Phát, sinh viên lớp K67I-CS3. Tôi xin cam đoan khóa luận *"Suy luận nhân quả trên dữ liệu chuỗi thời gian"* là công trình nghiên cứu do bản thân tôi thực hiện. Các nội dung nghiên cứu, kết quả trong khoá luận là xác thực.

Trong quá trình thực hiện khóa luận, tôi có sử dụng các công cụ Trí tuệ nhân tạo (AI) như một phương tiện hỗ trợ trong việc: rà soát lỗi cú pháp, tối ưu hóa phong cách viết trong báo cáo, và hỗ trợ chuyển đổi định dạng tài liệu. Tôi cam đoan rằng các công cụ này chỉ đóng vai trò hỗ trợ kỹ thuật.

Các thông tin sử dụng trong khoá luận là có cơ sở và không có nội dung nào sao chép từ các tài liệu mà không ghi rõ trích dẫn tham khảo. Tôi xin chịu trách nhiệm về lời cam đoan này.

Nếu vi phạm những điều cam đoan trên, tôi xin hoàn toàn chấp nhận mọi hình thức kỷ luật và truy cứu trách nhiệm theo quy định của Trường Đại học Công nghệ – ĐHQGHN.

Hà Nội, ngày tháng năm

Sinh viên

Nguyễn Thành
Phát

Tóm tắt

Suy luận nhân quả trên dữ liệu chuỗi thời gian là một lĩnh vực then chốt trong khoa học dữ liệu, đóng vai trò quan trọng trong việc hiểu rõ cơ chế vận hành của các hệ thống phức tạp thay vì chỉ dừng lại ở phân tích tương quan thống kê [1]. Trong báo cáo này, tôi trình bày nghiên cứu về bài toán xác định cấu trúc đồ thị có hướng không chu trình (Directed Acyclic Graph - DAG) từ dữ liệu chuỗi thời gian đa biến dựa trên bối cảnh cuộc thi do ADIA Lab phối hợp cùng CrunchDAO tổ chức.

Bài toán tập trung vào việc xác định mối quan hệ nhân quả giữa hai biến đặc biệt: biến can thiệp (X) và biến kết quả (Y), đồng thời phân loại vai trò của các biến còn lại trong hệ thống. Các biến này được phân định vào tám danh mục chức năng cụ thể bao gồm: biến gây nhiễu (Confounder), biến điều tiết (Collider), biến trung gian (Mediator), biến độc lập, và các biến thuộc chuỗi hệ quả hoặc nguyên nhân của X và Y [1]. Đây là một thách thức lớn do sự hiện diện của các yếu tố nhiễu và cấu trúc phụ thuộc phức tạp trong dữ liệu thực tế.

Để giải quyết bài toán này, tôi khảo sát và áp dụng các phương pháp suy luận nhân quả hiện đại, bao gồm cả hướng tiếp cận không giám sát (unsupervised) dựa trên kiểm định tính độc lập điều kiện và hướng tiếp cận có giám sát (supervised) tận dụng sức mạnh của các mô hình học máy. Quá trình thực nghiệm được thực hiện trên tập dữ liệu quy mô lớn gồm 47,000 mẫu được cung cấp bởi nền tảng CrunchDAO [1], với số lượng biến biến thiên nhằm kiểm chứng tính bền vững và khả năng tổng quát hóa của các mô hình.

Kết quả nghiên cứu được đánh giá dựa trên chỉ số độ chính xác cân bằng đa lớp (Multiclass Balanced Accuracy), tập trung vào khả năng nhận diện chính xác vai trò logic của các nút mạng đối với cặp biến $X \rightarrow Y$. Nghiên cứu không chỉ dừng lại ở việc dự đoán sự tồn tại của các cạnh mà còn hướng tới việc tinh chỉnh mô hình để giảm thiểu sai số trong việc xác định cấu trúc hệ thống, từ đó cung cấp một quy trình hiệu quả cho việc khám phá tri thức từ dữ liệu chuỗi thời gian.

Từ khóa: Suy luận nhân quả (Causal Discovery), Chuỗi thời gian (Time Series), Đồ thị có hướng không chu trình (DAG), CrunchDAO [1].

Abstract

Causal discovery on time series data is a pivotal field in data science, playing a crucial role in understanding the underlying mechanisms of complex systems rather than merely relying on statistical correlation analysis [1]. In this report, I present a study on the problem of discovering Directed Acyclic Graph (DAG) structures from multivariate time series data, based on the competition organized by ADIA Lab in collaboration with CrunchDAO.

The problem focuses on identifying the causal relationship between two primary variables: the treatment (X) and the outcome (Y), while simultaneously classifying the roles of the remaining variables within the system. These variables are categorized into eight functional roles, including: confounders, colliders, mediators, independent variables, and variables belonging to the causal or consequential chains of X and Y [1]. This task presents a significant challenge due to the presence of noise and the complex dependency structures inherent in real-world time series data.

To address this problem, I investigate and apply modern causal discovery methods, encompassing both unsupervised approaches based on conditional independence testing and supervised approaches leveraging the power of machine learning models. The experimental process was conducted on a large-scale dataset comprising 47,000 samples provided by the CrunchDAO platform [1], with varying numbers of variables to verify the robustness and generalization capabilities of the models.

Research results were evaluated using the Multiclass Balanced Accuracy metric, focusing on the ability to accurately identify the logical roles of nodes relative to the $X \rightarrow Y$ pair. This study goes beyond merely predicting the existence of edges, aiming to refine models to minimize errors in identifying the system's logical structure, thereby providing an effective framework for discovering causal knowledge from time series data.

Keywords: Causal Discovery, Time Series, Directed Acyclic Graph (DAG), CrunchDAO [1].

Contents

Lời cảm ơn

Lời cam đoan i

Tóm tắt ii

Abstract iii

List of Figures viii

List of Tables ix

Danh mục các từ viết tắt x

1 Introduction 1

1.1 Problem Statement 1

1.2 Motivation and Challenges 1

1.3 Objectives and Scope 3

1.4 Contributions 3

1.5 Thesis Structure 4

2 Background and Related Work 6

2.1 Causal Graph Theory 6

2.1.1 Directed Acyclic Graphs and Structural Causal Models 6

2.1.2 d-Separation and the Three Fundamental Motifs 6

2.2 The Eight-Class Causal Role Classification Problem 8

2.2.1 Problem Setup from the ADIA Lab Causal Discovery Challenge 8

2.2.2 Definition of the Eight Variable Roles 8

2.2.3 Challenges in Distinguishing Between Classes 9

2.3 Edge Tensor Representation 9

2.3.1 Motivation for the Edge-Based Approach 9

2.3.2	The Eight Channels of the Edge Tensor	10
2.4	Core Deep Learning Architectural Components	11
2.4.1	1D Convolutional Networks and Residual Blocks	11
2.4.2	Self-Attention Mechanism	13
2.4.3	Loss Function and Training Strategy	13
2.5	Two-Dimensional Density Estimation	14
2.5.1	2D Histogram and Joint Density	14
2.5.2	Gaussian Smoothing of Density Maps	14
2.5.3	Scatter Density as a Visual Conditional Independence Test . . .	14
2.6	Two-Dimensional Convolutional Networks	15
2.6.1	2D Convolution Operation	15
2.6.2	Hierarchical Spatial Feature Extraction	15
2.6.3	Why Conv2D Rather Than Flattening	16
	Gap Analysis	16
2.7	Existing Approaches on the ADIA Lab Challenge	17
2.7.1	First-Place Solution: End-to-End Deep Learning	17
2.7.2	Rank 3–5 Solutions: Feature Engineering with Gradient Boosting	17
2.7.3	Position of This Thesis	17
3	Proposed Method for Causal Role Classification	18
3.1	Architecture Overview	18
3.2	Preprocessing and Edge Tensor Construction	19
3.2.1	Data Normalization	19
3.2.2	Complete Directed Graph and Edge Pairs	19
3.2.3	8-Channel Edge Tensor	21
3.3	Edge Context Encoder	23
3.3.1	Per-Edge Feature Extraction via Conv1D	23
3.3.2	Edge Type Embedding and Merge	23
3.3.3	Binary Edge Head	23
3.4	Structural Attention Bias	24
3.4.1	Motivation	24
3.4.2	Six Topological Relationship Types	24
3.4.3	Learned Scalar Bias per Topological Type	25

3.4.4 Why This Succeeds Where Other Architectural Modifications Failed	25
3.5 Node Visual Representation	26
3.5.1 Motivation	26
3.5.2 8-Channel Scatter Density Image	26
3.5.3 Hierarchical Conv2D Feature Extractor	27
3.6 Fusion and Node Classification	28
3.6.1 Combining Edge Context with Visual Representation	28
3.6.2 Necessity of Edge Context	28
3.7 Loss Function and Training Objective	29
3.7.1 Dual-Head Loss with Lambda Weighting	29
3.7.2 Rationale for $\lambda = 0.7$	29
3.8 X/Y Remap Data Augmentation	30
3.8.1 Strategy	30
3.8.2 Efficient Implementation	30
3.9 Training Configuration	31
Chapter 3 Summary	31
4 Evaluation Methodology	32
4.1 Dataset	32
4.1.1 Data Source	32
4.1.2 Label Derivation from Adjacency Matrix	32
4.1.3 Dataset Characteristics	32
4.2 Evaluation Metric: Balanced Accuracy	33
4.3 Experimental Baselines	34
4.3.1 External Baselines	34
4.3.2 Internal Ablation Baselines	34
4.4 Experimental Setup	35
4.4.1 Hardware Environment	35
4.4.2 Two-Tier Evaluation Procedure	35
4.4.3 Experiment Design Principles	35
4.5 Complete Experiment Summary	35
Chapter 4 Summary	36
5 Results and Analysis	38

5.1	Deep Learning vs. Machine Learning: A Fair Comparison	38
5.1.1	v10: LightGBM Baseline	38
5.1.2	v13: Full ML Fullstack	38
5.2	Feature Engineering Contributions	40
5.2.1	Multi-Bandwidth Kernel Regression (+1.48%)	40
5.2.2	ANM Residual Channels (+1.50% over v5m)	40
5.2.3	XY Augmentation (+3.32% on v8b, +1.55% on v11)	40
5.3	Structural Attention Bias	41
5.3.1	Result and Mechanism (+2.65%)	41
5.3.2	Contrast with Failed Architectural Experiments	41
5.4	2D Scatter Density Pipeline	41
5.4.1	Edge Context Necessity	41
5.4.2	1D vs. 2D Representation	42
5.4.3	Channel Count: 12ch vs. 8ch	43
5.4.4	Loss Lambda Tuning	43
5.4.5	Final Model: v26e 8ch + XY Augmentation	44
5.5	Full Ablation Table	44
5.6	Synthesis and Lessons Learned	45
5.6.1	Shape Information Cannot Be Recovered After Scalar Compression	45
5.6.2	Structured Inductive Bias Outperforms Unconstrained Capacity	45
5.6.3	1D Curves and 2D Density Images Are Complementary	46
5.6.4	Edge Context Is the Foundation, Not an Auxiliary	46
	Chapter 5 Summary	46
	Conclusion	48
	References	49

List of Figures

1.1	Illustration of the eight causal roles of variable K with respect to the relationship $X \rightarrow Y$: each role corresponds to a distinct local DAG structure.	2
2.1	The three fundamental d-separation motifs: Chain ($X \rightarrow K \rightarrow Y$), Fork ($X \leftarrow K \rightarrow Y$), and Collider ($X \rightarrow K \leftarrow Y$). Arrows denote causal direction; $\perp$ and $\not\perp$ denote conditional independence and dependence respectively.	7
2.2	Summary of the eight causal roles and their corresponding DAG structures. Color-coded class pairs share similar statistical signatures, making classification harder.	10
2.3	Illustration of edge tensor construction for variable pair (u, v) : the three original channels represent the marginal distribution of u , the rank relationship between u and v , and the nonlinear kernel regression fit.	12
3.1	The architecture overview	20
3.2	The edge tensor demenstration	22

List of Tables

3.1	Six topological relationship types between edge pairs used in the structural attention bias.	24
4.1	Label derivation rules from the adjacency matrix for variable K	33
4.2	Complete experiment results. “Local” = 1,880-sample local test set; “LB” = CrunchDAO public leaderboard; “Private” = hidden test set (final submission only). $\Delta v2$ = change relative to reimplementat baseline. Dashes indicate not submitted or scored.	37
5.1	Summary comparison of deep learning and machine learning approaches.	39
5.2	Ablation: necessity of edge context for node classification. All variants use the same 8-channel node images; only the source of graph context differs.	42
5.3	Per-class recall: v11 (1D edge pipeline only) vs. v26e 8ch (1D + 2D fusion), base dataset without XY augmentation.	42
5.4	Effect of loss weight λ on v26e 8ch. $\mathcal{L} = \lambda \cdot \mathcal{L}_{\text{edge}} + (1 - \lambda) \cdot \mathcal{L}_{\text{node}}$	43
5.5	Per-class recall of the final proposed model (v26e 8ch + XY aug) on the local test set.	44
5.6	Incremental ablation: contribution of each component, where each row adds one component to the previous configuration.	45

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
BiLSTM	Bidirectional Long Short Term Memory	Mô hình bộ nhớ dài hạn ngắn hạn hai chiều
BO	Buffer Overflow	Lỗi tràn bộ đệm

Chapter 1

Introduction

1.1 Problem Statement

In data science and statistics, understanding *causal relationships* between variables is far more important than merely measuring correlation. Correlation can arise from many sources – hidden common causes (confounders), mediation, or pure statistical coincidence – whereas causal relationships reflect the true underlying mechanism of a system [2]. Correctly identifying causal structure is foundational to many practical problems: experimental design, policy impact evaluation, financial risk analysis, and causal inference in medicine [3].

The problem studied in this thesis is grounded in the **ADIA Lab Causal Discovery Challenge** [1, 4], a data science competition organized by ADIA Lab in collaboration with CrunchDAO. Specifically, the task is: given a dataset of $n = 1000$ observations over $p \in \{3, \dots, 10\}$ variables, where a causal edge $X \rightarrow Y$ always exists (with X the treatment variable and Y the outcome), **classify the role** of each remaining variable K into one of **8 classes**: Confounder, Mediator, Collider, Independent, Cause of X, Consequence of X, Cause of Y, or Consequence of Y.

This is a *supervised multiclass classification* problem: given 47,000 training datasets, the model must learn statistical features and latent graph structure to distinguish among eight roles – several of which share very similar statistical signatures and are inherently difficult to separate. Performance is measured by *Multiclass Balanced Accuracy* [4].

1.2 Motivation and Challenges

Although many classical causal inference algorithms have been proposed (PC, LiNGAM, NOTEARS, etc.), these methods typically aim to recover the *full DAG structure* and struggle when the goal is to precisely classify *the role of each individual variable* from finite-sample observational data [5].



Figure 1.1: Illustration of the eight causal roles of variable K with respect to the relationship $X \rightarrow Y$: each role corresponds to a distinct local DAG structure.

The ADIA Lab problem poses several specific challenges. First, **distinguishing between similar class pairs**: for example, Confounder ($K \rightarrow X, K \rightarrow Y$) and Cause of X ($K \rightarrow X$, no $K \rightarrow Y$) both induce correlation between K and X ; similarly, Mediator ($X \rightarrow K \rightarrow Y$) and Consequence of X ($X \rightarrow K$, no effect on Y) both exhibit $X \rightarrow K$ – the distinction lies in very subtle signals within the data. Second, **class imbalance**: the eight classes are not uniformly distributed, with Mediator and Collider having significantly fewer samples than others, causing models to neglect minority classes without appropriate handling strategies. Third, **permutation invariance**: the number of variables p varies from 3 to 10, and variable naming order carries no semantic meaning – the model must be invariant to variable permutation. Fourth, **nonlinear relationships**: the dependencies between variables are not necessarily linear, and simple statistics such as Pearson correlation are often insufficient to distinguish roles.

Classical solutions provided by the competition organizers (PC algorithm, Random Forest, deep learning baseline) achieved only around **40%** Balanced Accuracy [6] – not far above random guessing at 12.5% (1/8). The competition’s first-place solution [7] addressed these challenges through an end-to-end deep learning architecture: representing each variable pair as a *3-channel edge tensor* (incorporating nonlinear kernel regression features), then using residual Conv1D and self-attention to learn graph-wide representations and classify each variable’s role, achieving **76.70%** Balanced Accuracy [6]. This thesis reimplements that solution, analyzes its components, and proposes extensions to an 8-channel feature representation combined with X/Y remap data augmentation, ultimately achieving **80.82%** on the public leaderboard – surpassing the first-place solution.

1.3 Objectives and Scope

This thesis has two primary objectives. First, to **reimplement and analyze** the first-place solution of the ADIA Lab Causal Discovery Challenge: understanding each architectural component, the training procedure, and the rationale behind each design choice. This objective produces a solid, reproducible baseline. Second, to **propose improvements** aimed at increasing Balanced Accuracy, particularly on difficult classes (Mediator, Collider), through additional feature channels, architectural refinements, and training strategies.

Scope: The thesis focuses on i.i.d. observational data following the ADIA Lab setup (not time-series with autocorrelation). The model is implemented and evaluated on the CrunchDAO platform using the official Multiclass Balanced Accuracy metric.

1.4 Contributions

The specific contributions of this thesis are as follows.

Reimplementation and detailed analysis of the first-place solution: The complete pipeline is reimplemented – from edge tensor construction (3 channels, kernel regression) to the residual Conv1D + self-attention + dual-head loss architecture – achieving **73.96%** (local) / **73.61%** (public leaderboard), confirming the reproducibility of the original solution.

Extension of edge tensors to 8 channels: Three multi-bandwidth kernel regression channels ($h \in \{0.2, 0.5, 1.0\}$) and three multi-bandwidth ANM (Additive Noise Model) residual channels are added, enhancing the model’s ability to detect causal asymmetry and distinguish difficult classes such as Collider and Consequence of X.

X/Y remap data augmentation strategy: A data augmentation technique is proposed and implemented by treating each directed edge $A \rightarrow B$ in the ground-truth DAG as a new treatment-outcome pair (X', Y') , recomputing 8-class labels for all remaining variables relative to the new pair, yielding approximately $11.2\times$ additional training samples.

Structural attention bias: A topology-aware attention bias is introduced into the self-attention layers, encoding prior knowledge about causal relationships between edge pairs (chain, fork, collider) directly into the architecture with minimal additional parameters.

Results surpassing the first-place solution: The best proposed model (v11 + XY augmentation) achieves **81.14%** (local) / **80.82%** (public leaderboard), surpassing the competition’s first-place solution (**76.70%**) by a margin of $+4.12\%$ [6].

1.5 Thesis Structure

The remainder of the thesis is organized as follows.

Chapter 2 – Background: Presents causal graph theory, definitions of the eight roles, edge tensor features, and the core deep learning architectural components used.

Chapter 3 – Methodology: Describes in detail the model architecture, feature construction pipeline, loss function, and training strategy, along with the proposed improvements.

Chapter 4 – Experiments: Presents the experimental setup, evaluation procedure, and comparative results between the baseline and proposed improvements.

Chapter 5 – Results and Analysis: Provides detailed analysis of all experimental results, ablation studies, error analysis, and key lessons learned.

Chapter 5.6.4 – Conclusion: Summarizes the results achieved, discusses limi-

tations, and outlines directions for future work.

Chapter 2

Background and Related Work

This chapter presents the theoretical foundations directly relevant to the problem and solution developed in the thesis: (1) causal graph theory and the definition of the eight variable roles, (2) statistical features used to represent observational data as edge tensors, (3) core deep learning architectural components (residual Conv1D, self-attention), and (4) an overview of existing approaches on the ADIA Lab Causal Discovery Challenge.

2.1 Causal Graph Theory

2.1.1 Directed Acyclic Graphs and Structural Causal Models

Causal relationships between random variables are represented using a *Directed Acyclic Graph* (DAG) $G = (V, E)$, where each edge $X_i \rightarrow X_j$ denotes a direct causal effect from X_i to X_j . Combined with structural equations

$$X_i := f_i(\text{Pa}(X_i), U_i), \quad (2.1)$$

this forms a *Structural Causal Model* (SCM) [2], where $\text{Pa}(X_i)$ is the set of direct parents of X_i and U_i is an independent exogenous noise term. SCMs clearly distinguish between the observational distribution $P(V)$ and the interventional distribution $P(V \mid \text{do}(X = x))$, which is the foundation for defining the causal meaning of each relationship in the graph [2].

2.1.2 d-Separation and the Three Fundamental Motifs

Conditional independence on a DAG is derived via the *d-separation* criterion [8]: a set Z d-separates X and Y if every path between X and Y is blocked by Z . Under the Faithfulness assumption (every statistical independence reflects a corresponding graphical separation [5]), one can reason from data back to graph structure.

Three fundamental motifs are directly relevant to classifying the role of variable K .

Chain: $X \rightarrow K \rightarrow Y$. Conditioning on K blocks the path: $X \perp Y \mid K$. This is the

structure of the **Mediator** role. **Fork (Confounder)**: $X \leftarrow K \rightarrow Y$. Conditioning on K blocks the path: $X \perp Y \mid K$. This is the structure of the **Confounder** role. **Collider**: $X \rightarrow K \leftarrow Y$. Marginally: $X \perp Y$; but conditioning on K induces dependence: $X \not\perp Y \mid K$ (collider bias). This is the structure of the **Collider** role.

Note that Chain and Fork produce the same marginal statistical signature ($X \perp Y \mid K$), while Collider produces the opposite. This leads to fundamental difficulty in distinguishing Mediator from Confounder using conditional independence tests alone.

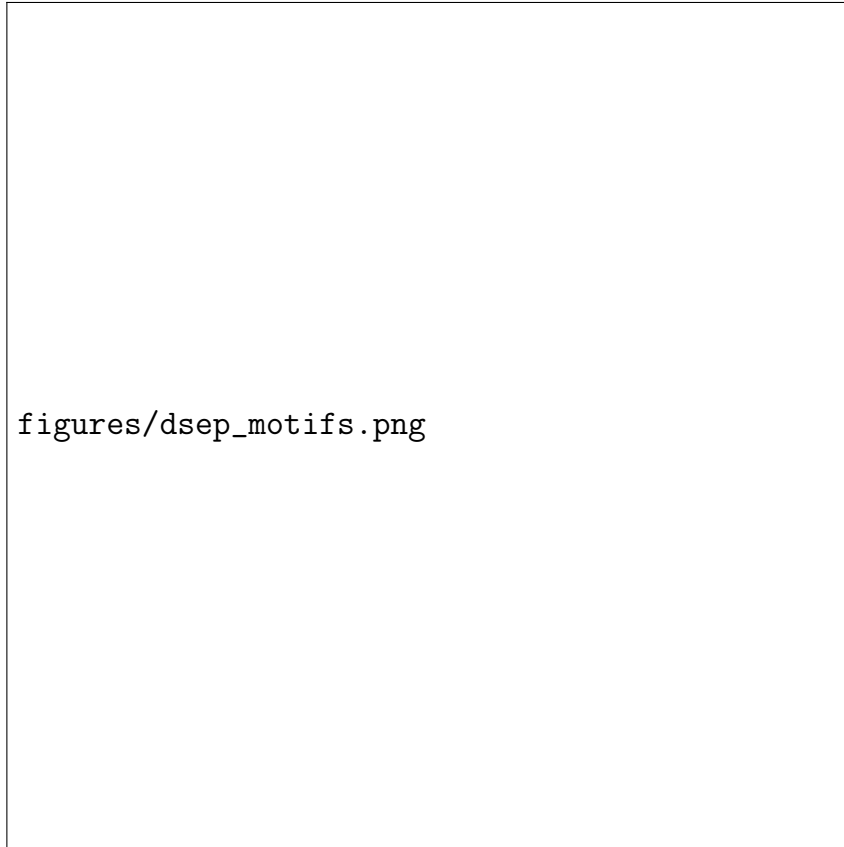


Figure 2.1: The three fundamental d-separation motifs: Chain ($X \rightarrow K \rightarrow Y$), Fork ($X \leftarrow K \rightarrow Y$), and Collider ($X \rightarrow K \leftarrow Y$). Arrows denote causal direction; $\perp$ and $\not\perp$ denote conditional independence and dependence respectively.

2.2 The Eight-Class Causal Role Classification Problem

2.2.1 Problem Setup from the ADIA Lab Causal Discovery Challenge

The ADIA Lab Causal Discovery Challenge [4, 9] poses the following problem: given a dataset of $n = 1000$ observations over $p \in \{3, \dots, 10\}$ variables, where two special variables X (treatment) and Y (outcome) are identified and the causal edge $X \rightarrow Y$ is assumed to exist, classify the role of each variable $K \neq X, K \neq Y$ with respect to the relationship $X \rightarrow Y$.

This is a **multiclass classification** problem: for each tuple $(X, Y, K, \mathcal{D})$, the model predicts one of 8 labels. There are 47,000 training datasets in total. The official evaluation metric is *Multiclass Balanced Accuracy*:

$$\text{BA} = \frac{1}{C} \sum_{c=1}^C \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}, \quad (2.2)$$

with $C = 8$ classes. This metric handles class imbalance naturally – each class is weighted equally regardless of sample count [4, 6]. A random guessing strategy achieves 12.5% (1/8), and the organizer-provided baselines reach approximately 40% [6].

2.2.2 Definition of the Eight Variable Roles

The eight labels are defined according to the causal structure of K in the DAG containing X , Y , and K (with the edge $X \rightarrow Y$ always assumed present):

1. **Confounder:** $K \rightarrow X$ and $K \rightarrow Y$. Variable K is a common cause, inducing spurious correlation between X and Y if K is not controlled for.
2. **Mediator:** $X \rightarrow K \rightarrow Y$ (a mediation pathway parallel to the direct edge $X \rightarrow Y$). Variable K transmits part of the causal effect from X to Y .
3. **Collider:** $X \rightarrow K \leftarrow Y$. Both X and Y are causes of K . Conditioning on K induces spurious dependence between X and Y .
4. **Independent:** K has no causal edges to or from X or Y . K is statistically independent of both.

5. **Cause of X:** $K \rightarrow X$, but there is no direct path between K and Y other than through X .
6. **Consequence of X:** $X \rightarrow K$, and K has no effect on Y .
7. **Cause of Y:** $K \rightarrow Y$, but there is no direct path between K and X other than through Y .
8. **Consequence of Y:** $Y \rightarrow K$, and K has no effect on X .

2.2.3 Challenges in Distinguishing Between Classes

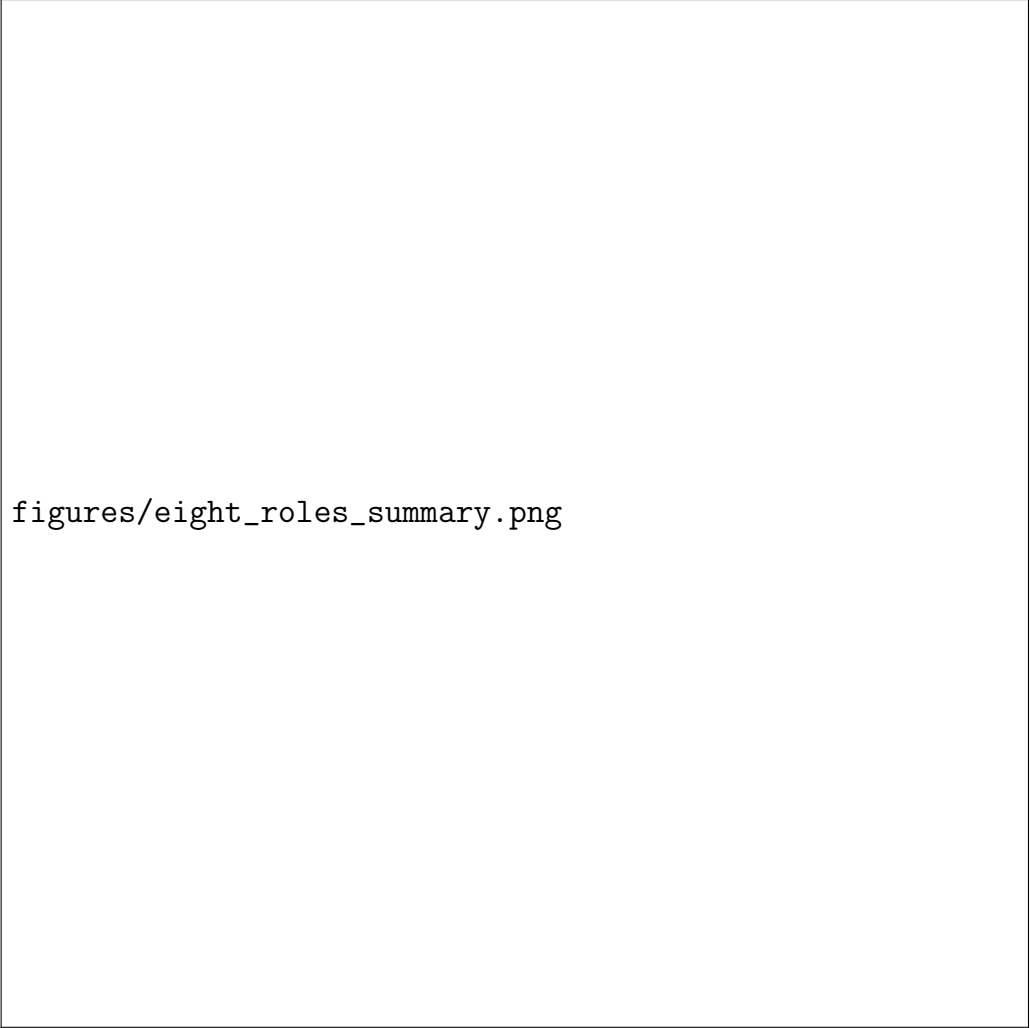
Several pairs of labels share similar statistical signatures, making classification difficult. **Confounder vs. Cause of X:** Both have $K \rightarrow X$, creating correlation between K and X . The difference lies in whether K has an edge to Y . **Mediator vs. Consequence of X:** Both have $X \rightarrow K$. The difference lies in whether K subsequently affects Y . **Mediator vs. Confounder:** As analyzed above, both produce $X \perp Y \mid K$ under d-separation, but through entirely different causal mechanisms.

According to the post-competition analysis by the organizers [6], even the first-place solution achieved only 65.67% on the Mediator class – the lowest among all eight classes – while achieving 86.18% on Independent. The organizers’ supervised baseline reached only 13.89% on Mediator and 18.60% on Confounder, confirming these as the two most challenging classes [6]. In the reimplementation results of this thesis, the two hardest classes are Mediator ($\sim 56\%$) and Collider ($\sim 63\%$).

2.3 Edge Tensor Representation

2.3.1 Motivation for the Edge-Based Approach

Rather than processing a raw data matrix ($n \times p$), the first-place solution [7] represents each *ordered variable pair* (u, v) as a multi-channel **edge tensor** of length n . This approach has two advantages: (1) the architecture becomes *permutation-equivariant* with respect to variable ordering; and (2) directional information for $(u \rightarrow v)$ and $(v \rightarrow u)$ is encoded separately, allowing the model to learn causal asymmetry.



figures/eight_roles_summary.png

Figure 2.2: Summary of the eight causal roles and their corresponding DAG structures. Color-coded class pairs share similar statistical signatures, making classification harder.

In this thesis, the edge tensor is extended from 3 channels (original solution) to **8 channels**, adding multi-bandwidth kernel regression and multi-bandwidth ANM residual channels.

2.3.2 The Eight Channels of the Edge Tensor

Given variable pair (u, v) and permutation σ that sorts u in ascending order, the 8-channel edge tensor is constructed as follows.

Channel 1 – Sorted u . The sequence $\mathbf{c}_1 = (u_{\sigma(1)}, \dots, u_{\sigma(n)})$ preserves the marginal distribution of u .

Channel 2 – v sorted by rank of u . The sequence $\mathbf{c}_2 = (v_{\sigma(1)}, \dots, v_{\sigma(n)})$ encodes the monotonic relationship and rank correlation between u and v .

Channels 3–5 – Multi-bandwidth kernel regression. At each point $u_{\sigma(i)}$, the Nadaraya–Watson estimator [10, 11]:

$$\hat{m}_h(u_{\sigma(i)}) = \frac{\sum_{j=1}^n \exp\left(-\frac{(u_{\sigma(i)} - u_j)^2}{2h^2}\right) v_j}{\sum_{j=1}^n \exp\left(-\frac{(u_{\sigma(i)} - u_j)^2}{2h^2}\right)}, \quad (2.3)$$

is computed with three bandwidths $h \in \{0.2, 0.5, 1.0\}$, producing channels $\mathbf{c}_3, \mathbf{c}_4, \mathbf{c}_5$. Each bandwidth captures a different smoothness scale of the regression function – $h = 0.2$ is sensitive to local structure while $h = 1.0$ reflects global trends. Using multiple bandwidths prevents the model from depending on any single fixed choice of h .

Channels 6–8 – Multi-bandwidth ANM residuals. The Additive Noise Model (ANM) [12] assumes $V = f(U) + \varepsilon$ with $\varepsilon \perp U$. The regression residual in direction $u \rightarrow v$ at point $u_{\sigma(i)}$ is:

$$r_h(u_{\sigma(i)}) = v_{\sigma(i)} - \hat{m}_h(u_{\sigma(i)}), \quad (2.4)$$

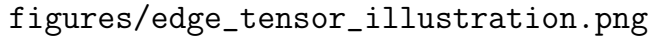
computed with the same three bandwidths, producing channels $\mathbf{c}_6, \mathbf{c}_7, \mathbf{c}_8$.

The causal asymmetry of ANM residuals is what makes this channel informative: in a model $U \rightarrow V$ with non-Gaussian noise, the residual $V - \hat{m}(U)$ is closer to independent of U than the reverse direction $U - \hat{m}(V)$ [12]. The Conv1D model can learn this signature from the distributional shape of $\mathbf{c}_{6..8}$. Experiments show that ANM channels improve Collider accuracy by +6.1% and Consequence of X by +5.0% compared to the 3-channel baseline.

2.4 Core Deep Learning Architectural Components

2.4.1 1D Convolutional Networks and Residual Blocks

1D convolutional networks apply learned filters over 1D sequences, making them well-suited to extract local features from edge tensors (each edge is a sequence of



figures/edge_tensor_illustration.png

Figure 2.3: Illustration of edge tensor construction for variable pair (u, v) : the three original channels represent the marginal distribution of u , the rank relationship between u and v , and the nonlinear kernel regression fit.

length $n = 1000$). With a filter of width k , a Conv1D layer computes:

$$y_t = \sum_{j=0}^{k-1} w_j \cdot x_{t+j} + b, \quad (2.5)$$

yielding a feature representation at each position in the input sequence.

To avoid vanishing gradients as network depth increases, *skip connections* (residual connections) [13] are used:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}; \{W_i\}) + \mathbf{x}, \quad (2.6)$$

where $\mathcal{F}$ is the learned mapping and $\mathbf{x}$ is the shortcut input. Combined with *Group Normalization* [14] (replacing Batch Normalization for stability with small batch

sizes) and the GELU activation function [15], each residual block in the model takes the form:

$$\mathbf{y} = \mathbf{x} + \text{Conv1D}(\text{GELU}(\text{GroupNorm}(\mathbf{x}))). \quad (2.7)$$

2.4.2 Self-Attention Mechanism

After each edge tensor is processed through Conv1D blocks and reduced to a representation vector $\mathbf{e}_{uv} \in \mathbb{R}^d$ via Global Average Pooling, information from all edges in the graph must be aggregated to reason about each variable’s role. The *self-attention* mechanism [16] allows each element in a sequence to query information from all other elements:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad (2.8)$$

where $Q = \mathbf{X}W^Q$, $K = \mathbf{X}W^K$, $V = \mathbf{X}W^V$ are linear projections.

Multi-head attention [16] runs H attention heads in parallel:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W^O. \quad (2.9)$$

Two layers of multi-head self-attention are placed after the pooling step to aggregate edge-level representations across all edges in a dataset, enabling the model to compare graph-wide information and transition from edge-level to node-level representations.

2.4.3 Loss Function and Training Strategy

The model uses two output heads with two loss functions. The **Node head** (8-class) is the primary loss: Cross-Entropy (CE) with inverse-frequency class weights. The **Edge head** (binary – causal edge present or not) is an auxiliary loss used only during training. The combined loss is:

$$\mathcal{L} = \mathcal{L}_{\text{node}} + \lambda \cdot \mathcal{L}_{\text{edge}}, \quad (2.10)$$

where λ is a balancing hyperparameter. Class weights in CE are set inversely proportional to class frequency in the training set to handle class imbalance. Optimization uses *AdamW* [17] with a *cosine annealing* schedule [18].

2.5 Two-Dimensional Density Estimation

The Node Visual Classifier (Section 3.5) represents each variable K as a two-dimensional scatter density image capturing the joint distribution of K with X and Y . This section introduces the tools required to construct and interpret such images.

2.5.1 2D Histogram and Joint Density

Given n observation pairs (a_i, b_i) for two variables a and b , a *two-dimensional histogram* partitions the joint domain $[\min_a, \max_a] \times [\min_b, \max_b]$ into a $G \times G$ grid of equal-width bins and counts the number of observations falling in each bin:

$$H[r, c] = \#\{i : a_i \in \text{bin}_r, b_i \in \text{bin}_c\}, \quad r, c \in \{1, \dots, G\}. \quad (2.11)$$

This matrix $H \in \mathbb{Z}^{G \times G}$ is a discrete approximation to the joint probability mass function $p(a, b)$.

2.5.2 Gaussian Smoothing of Density Maps

Raw histograms are noisy at finite sample size n . To produce a continuous, visually interpretable density image, each histogram is convolved with a Gaussian kernel of standard deviation σ (in bin units):

$$\hat{H}_\sigma[r, c] = \sum_{r', c'} H[r', c'] \cdot \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(r - r')^2 + (c - c')^2}{2\sigma^2}\right). \quad (2.12)$$

The smoothed image $\hat{H}_\sigma \in \mathbb{R}^{G \times G}$ represents the estimated joint density $\hat{p}(a, b)$. Larger σ produces coarser but more stable density maps; smaller σ retains finer structure at the cost of increased noise sensitivity. In this thesis, a dual-sigma strategy is used: $\sigma_{\text{raw}} = 4.0$ for raw scatter channels and $\sigma_{\text{ANM}} = 2.0$ for ANM residual channels, for reasons discussed in Section 3.5.

2.5.3 Scatter Density as a Visual Conditional Independence Test

The central insight motivating the 2D node representation is that *conditional independence has a visual signature in scatter density images*. Under the Additive

Noise Model (ANM) assumption, if the true causal direction is $u \rightarrow v$, then the residual $r_{u \rightarrow v} = v - \hat{m}(u)$ is independent of u : the scatter plot of $(u, r_{u \rightarrow v})$ appears as a uniform horizontal band with no visible pattern. Conversely, the scatter plot of $(v, r_{v \rightarrow u})$ under the wrong causal direction exhibits systematic curvature or heteroscedasticity [19]. This directional asymmetry is precisely what the Conv2D network is trained to detect: the “flatness” versus “structure” of the residual density image carries the information about causal direction that is equivalent to a visual conditional independence test. Rather than computing a scalar independence measure – which discards spatial information – the model retains the full two-dimensional density and lets Conv2D learn the relevant visual test directly.

2.6 Two-Dimensional Convolutional Networks

The 1D convolutional architecture described in Section 2.4.1 processes sequential data along a single axis. Two-dimensional convolutional networks (Conv2D) extend this to spatial data with two axes, such as images or density maps.

2.6.1 2D Convolution Operation

A 2D convolutional layer applies a learned filter $\mathbf{W} \in \mathbb{R}^{k \times k}$ (kernel size k) to an input feature map $\mathbf{X} \in \mathbb{R}^{H \times W}$:

$$\mathbf{Y}[r, c] = \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} \mathbf{W}[i, j] \cdot \mathbf{X}[r + i, c + j] + b, \quad (2.13)$$

where b is a learned bias and the spatial dimensions of $\mathbf{Y}$ are determined by the stride and padding. Each filter detects a specific local spatial pattern; a stack of C_{out} filters produces C_{out} feature maps simultaneously.

2.6.2 Hierarchical Spatial Feature Extraction

Successive Conv2D layers learn increasingly abstract spatial representations. Early layers detect low-level textures such as local density peaks and gradients; deeper layers detect higher-level structural patterns such as the global shape of the joint distribution (uniform, banded, curved, heteroscedastic). Spatial downsampling via strided convolutions reduces the $H \times W$ resolution progressively ($32 \rightarrow 16 \rightarrow 8 \rightarrow 4$),

forcing the network to summarize spatial information at increasing scales before pooling to a fixed-size embedding. This hierarchy is well-suited to the scatter density images in this thesis, where the discriminating signal operates at multiple spatial scales simultaneously.

2.6.3 Why Conv2D Rather Than Flattening

An alternative to Conv2D would be to flatten the $G \times G$ density image into a G^2 -dimensional vector and process it with a fully connected network. This approach loses the spatial inductive bias: a fully connected layer treats all (r, c) positions as independent features and must learn spatial relationships from scratch, requiring far more data and parameters. Conv2D’s *local connectivity* and *translation equivariance* are appropriate inductive biases for density images, since the presence of a density peak at position (r, c) is structurally similar to a peak at $(r + \delta, c + \delta)$ and should be detected by the same filter regardless of exact position.

Gap Analysis: What Existing Approaches Leave Open

The review of existing approaches (Section 2.7) reveals two open gaps that motivate the contributions of this thesis.

Gap 1 – Irreversible scalar compression in ML approaches. Feature engineering approaches extract scalar statistics from the $n = 1000$ observations and discard the raw data. Correlation coefficients, partial correlations, and independence test statistics are each fixed-dimensional projections of the joint distribution, losing all information about the *shape* of the conditional expectation function. As shown in Chapter 5, even a 300-feature pipeline with four causal discovery algorithms cannot recover the performance of a model that operates on the full functional curves. This gap motivates the 8-channel edge tensor and the 2D density image representations, both of which preserve the distributional shape information that scalar summaries discard.

Gap 2 – Node-level visual context is absent in pure edge-level models. The first-place deep learning solution represents the problem entirely at the edge level: each variable pair (u, v) is processed independently, and node K ’s causal role is inferred by aggregating four edge embeddings. This aggregation discards

the joint visual pattern of K with respect to both X and Y *simultaneously*. A Confounder ($K \rightarrow X, K \rightarrow Y$) and a Collider ($X \rightarrow K, Y \rightarrow K$) have different joint scatter patterns in the (K, X) versus (K, Y) plane that are not fully captured by two separate 1D curves. This gap motivates the 2D scatter density node image pipeline introduced in Chapter 3.

2.7 Existing Approaches on the ADIA Lab Challenge

2.7.1 First-Place Solution: End-to-End Deep Learning

The first-place solution [7] is an end-to-end deep learning model with the following architecture: *Edge tensor construction* (3 channels) $\rightarrow$ *Stem Conv1D* (3 $\rightarrow$ 64 channels) $\rightarrow$ 5 *Residual Conv1D blocks* $\rightarrow$ *Global Average Pooling* $\rightarrow$ *Merge with edge-type embedding* $\rightarrow$ 2 *Self-Attention layers* $\rightarrow$ *Node head* (8-class) + *Edge head* (binary, auxiliary). The model has approximately 188K parameters and achieves **76.7%** Balanced Accuracy on the public leaderboard.

2.7.2 Rank 3–5 Solutions: Feature Engineering with Gradient Boosting

The rank-4 [20] and rank-5 [21] solutions follow a feature engineering + gradient boosting approach: extracting statistical features (Pearson/Spearman correlation, partial correlation, conditional mutual information, HSIC, regression residuals) together with signals from classical causal discovery algorithms (LiNGAM [12], GES), then using LightGBM/XGBoost ensembles for 8-class classification. Advantages include interpretability and no GPU requirement; disadvantages include dependence on manual feature engineering and the inability to learn representations end-to-end.

2.7.3 Position of This Thesis

This thesis reimplements and analyzes the first-place solution in detail, then proposes improvements to raise Balanced Accuracy – particularly on the difficult classes Mediator ($\sim 56\%$) and Collider ($\sim 63\%$) in the reimplementation baseline. The baseline score achieved in this thesis is **73.96%** local / **73.61%** public leaderboard. The specific technical contributions are presented in Chapter 3.

Chapter 3

Proposed Method for Causal Role Classification

This chapter describes the complete pipeline of the model proposed in this thesis. The method takes as input an observational dataset of $n = 1000$ observations over $p \in \{3, \dots, 10\}$ variables – with a designated treatment variable X and outcome variable Y – and produces an 8-class causal role label for each remaining variable $K \notin \{X, Y\}$.

The proposed model builds on the edge-centric deep learning framework introduced by the first-place solution of the ADIA Lab Causal Discovery Challenge [6, 7], and extends it with five targeted contributions: (1) multi-bandwidth kernel regression and ANM residual channels that expand the edge representation from 3 to 8 channels; (2) a topology-aware structural attention bias injected into self-attention layers; (3) a 2D scatter density node image pipeline that provides a complementary visual representation of each node’s relationship to X and Y ; (4) a λ -weighted dual-loss training objective that balances auxiliary edge supervision against the primary node classification objective; and (5) X/Y remap data augmentation that multiplies the effective training set by approximately $11\times$.

3.1 Architecture Overview

The model consists of two parallel processing pipelines that operate on different representations of the same observational data, followed by a fusion step for final classification, as illustrated in Figure ??.

The **Edge Context Encoder** (Section 3.3) processes all $p(p-1)$ directed variable pairs simultaneously. Each ordered pair (u, v) is represented as an 8-channel 1D tensor of length n , capturing the statistical and causal relationship between u and v through sorted observations, multi-bandwidth kernel regression curves, and ANM residuals. These tensors are processed independently by a shared-weight Conv1D network, then aggregated across the full graph via structural self-attention. A bi-

nary edge head supervises this pipeline during training, forcing the edge embeddings to encode causally meaningful graph structure.

The **Node Visual Classifier** (Section 3.5) processes each non- X/Y variable K individually. For each such node, an 8-channel 32×32 scatter density image is constructed by rendering the joint distribution of K against X and Y (and their ANM residuals) as smoothed 2D histograms. A hierarchical Conv2D network extracts visual features from this image.

The **Fusion** step (Section 3.6) combines the node’s 2D visual embedding with the four edge context embeddings corresponding to edges $K \rightarrow X$, $K \rightarrow Y$, $X \rightarrow K$, and $Y \rightarrow K$, producing the final 8-class prediction for each node.

3.2 Preprocessing and Edge Tensor Construction

3.2.1 Data Normalization

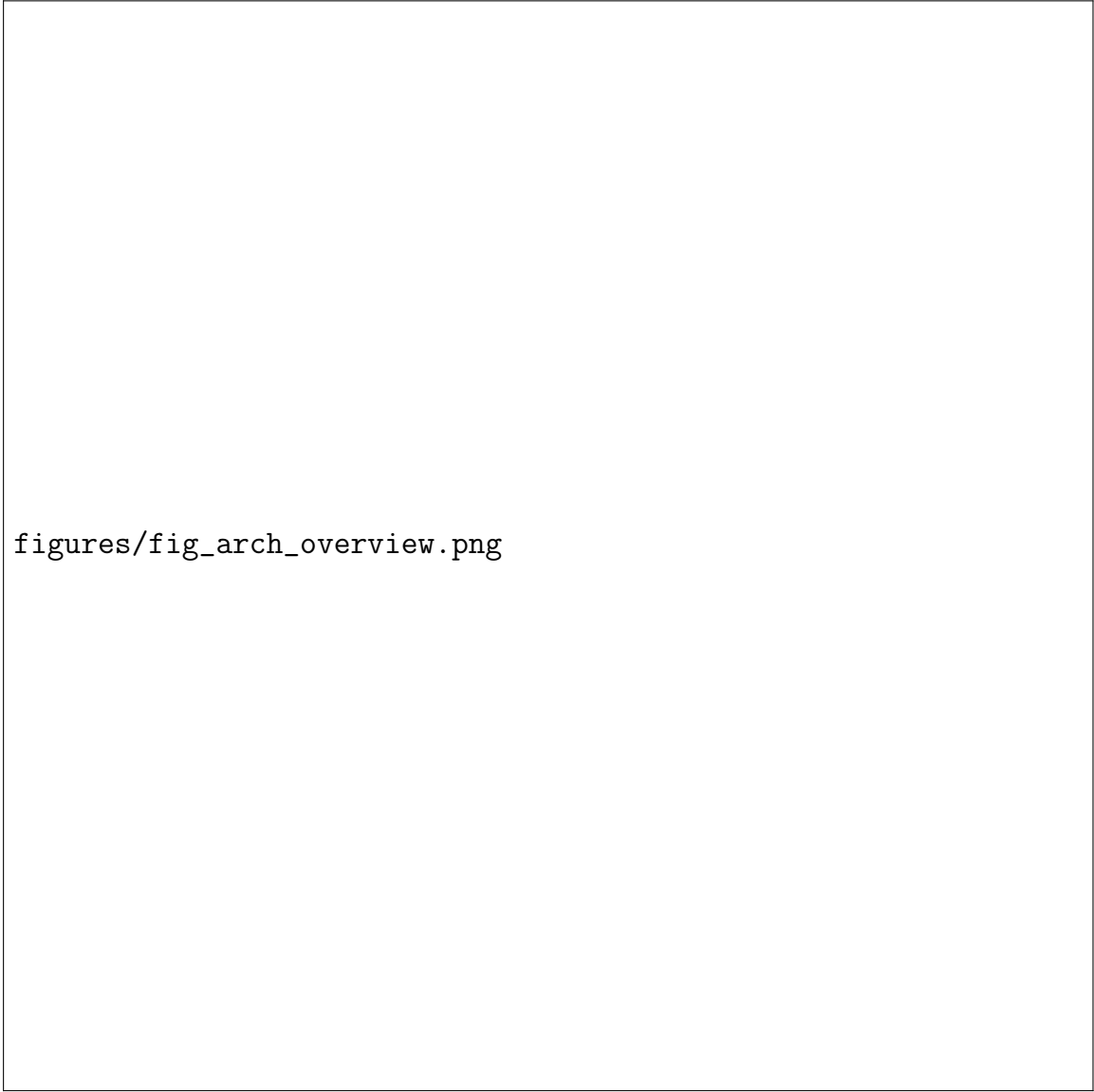
Each input dataset is a matrix $\mathbf{D} \in \mathbb{R}^{n \times p}$ with $n = 1000$ observations. Prior to feature construction, each column (variable) is standardized to zero mean and unit variance via z-score normalization:

$$\tilde{x}_i = \frac{x_i - \mu}{\sigma + \varepsilon}, \quad (3.1)$$

where μ and σ are the column mean and standard deviation respectively, and $\varepsilon = 10^{-8}$ prevents division by zero. Normalization ensures that all subsequent computations – particularly kernel bandwidth selection – are scale-invariant across variables and datasets.

3.2.2 Complete Directed Graph and Edge Pairs

From p variables, a complete directed graph with $p(p - 1)$ ordered pairs is constructed. Every ordered pair (u, v) with $u \neq v$ produces a separate edge tensor: the pairs (u, v) and (v, u) are treated as distinct, because the directional asymmetry between them is a primary causal signal. This permutation-equivariant design ensures that classification results are independent of the arbitrary naming order of variables.



figures/fig_arch_overview.png

Figure 3.1: The architecture overview

3.2.3 8-Channel Edge Tensor

For each ordered pair (u, v) , let σ denote the permutation that sorts variable u in ascending order, so that $u_{\sigma(1)} \leq \dots \leq u_{\sigma(n)}$. The 8-channel edge tensor $\mathbf{T}_{uv} \in \mathbb{R}^{8 \times n}$ is constructed as follows. Figure ?? illustrates the resulting curves for each channel.

Channels 1–2: Sorted observations.

$$\mathbf{c}_1 = (u_{\sigma(1)}, \dots, u_{\sigma(n)}), \quad \mathbf{c}_2 = (v_{\sigma(1)}, \dots, v_{\sigma(n)}). \quad (3.2)$$

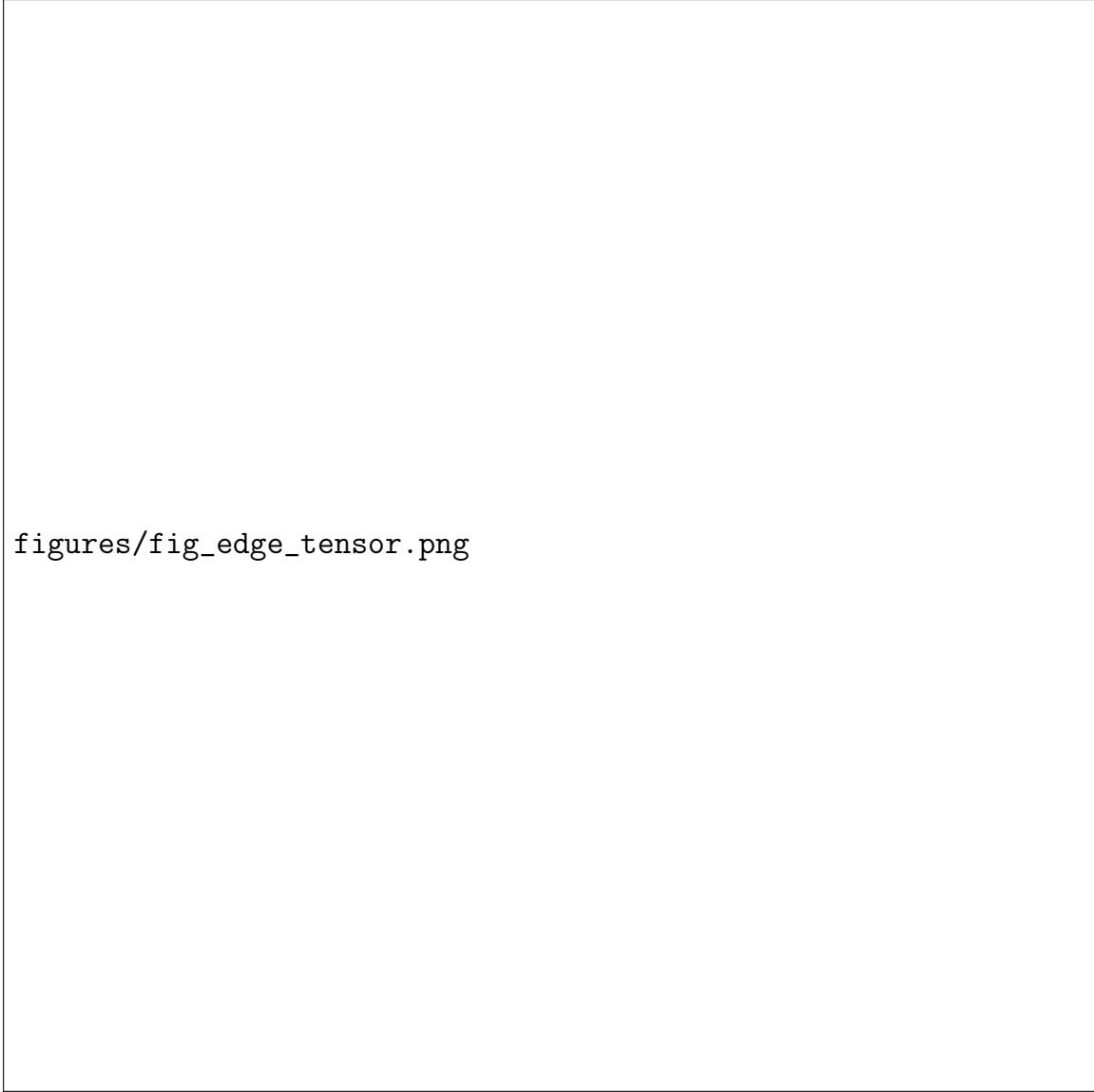
Channel 1 preserves the marginal distribution of u ; channel 2 encodes the functional relationship between u and v as an empirical curve. When sorted by rank of u , channel 2 traces the conditional expectation $\mathbb{E}[v \mid u]$: monotone relationships appear as smooth increasing or decreasing curves, non-monotone relationships as curved or U-shaped traces. This representation retains the full *shape* of the conditional relationship, which is information irreversibly lost by scalar summaries such as correlation coefficients.

Channels 3–5: Multi-bandwidth kernel regression. At each sorted point $u_{\sigma(i)}$, the Nadaraya–Watson estimator [10, 11] computes:

$$\hat{m}_h(u_{\sigma(i)}) = \frac{\sum_{j=1}^n \exp\left(-\frac{(u_{\sigma(i)} - u_j)^2}{2h^2}\right) v_j}{\sum_{j=1}^n \exp\left(-\frac{(u_{\sigma(i)} - u_j)^2}{2h^2}\right)}, \quad (3.3)$$

with three bandwidths $h \in \{0.2, 0.5, 1.0\}$, yielding channels $\mathbf{c}_3, \mathbf{c}_4, \mathbf{c}_5$. Each bandwidth captures a different smoothness scale of the nonlinear relationship: $h = 0.2$ is sensitive to local curvature and sharp transitions, while $h = 1.0$ reflects global monotone trends. Using all three bandwidths together prevents the model from depending on a single fixed scale assumption.

Channels 6–8: Multi-bandwidth ANM residuals. The Additive Noise Model (ANM) [19] posits that if $U \rightarrow V$ is the true causal direction, the residual $V - \hat{m}(U)$ should be approximately independent of U ; the reverse-direction residual $U - \hat{m}(V)$



figures/fig_edge_tensor.png

Figure 3.2: The edge tensor demonstration

will retain structure. This asymmetry is encoded in channels 6–8:

$$r_h(u_{\sigma(i)}) = v_{\sigma(i)} - \hat{m}_h(u_{\sigma(i)}), \quad h \in \{0.2, 0.5, 1.0\}, \quad (3.4)$$

yielding channels $\mathbf{c}_6, \mathbf{c}_7, \mathbf{c}_8$. When $u \rightarrow v$ holds, the residual curve is approximately flat (independent of u); under the reversed direction, the residual retains a systematic nonlinear shape. Conv1D learns to detect this difference directly from the curve shape.

3.3 Edge Context Encoder

3.3.1 Per-Edge Feature Extraction via Conv1D

The 8-channel edge tensor $\mathbf{T}_{uv} \in \mathbb{R}^{8 \times n}$ is processed by a shared-weight Conv1D network to produce a $d = 64$ dimensional edge embedding. The network consists of a stem layer followed by five residual blocks. The **stem layer** expands the 8 input channels to $d = 64$ hidden channels:

$$\mathbf{H}_0 = \text{Conv1D}_{8 \rightarrow 64}(\mathbf{T}_{uv}) \in \mathbb{R}^{64 \times n}. \quad (3.5)$$

Each of the five **residual Conv1D blocks** applies the pattern from He et al. [13], adapted with Group Normalization [14] for stability under small batch sizes and the GELU activation [15]:

$$\mathbf{H}_{l+1} = \mathbf{H}_l + \text{Conv1D}\left(\text{GELU}\left(\text{GroupNorm}(\mathbf{H}_l)\right)\right). \quad (3.6)$$

After the five residual blocks, **Global Average Pooling** (GAP) reduces the sequence dimension, producing a fixed-size representation regardless of n :

$$\mathbf{e}_{uv}^{\text{conv}} = \text{GAP}(\mathbf{H}_5) = \frac{1}{n} \sum_{t=1}^n \mathbf{H}_5[:, t] \in \mathbb{R}^{64}. \quad (3.7)$$

3.3.2 Edge Type Embedding and Merge

Each directed edge (u, v) is assigned a discrete **edge type** τ_{uv} from a 7-class taxonomy based on the positions of u and v relative to X and Y . A learnable embedding table maps each type to a vector $\mathbf{t}_{\tau_{uv}} \in \mathbb{R}^{64}$. The final edge representation combines the Conv1D embedding with the type embedding via addition, which preserves the representation dimension:

$$\tilde{\mathbf{e}}_{uv} = \mathbf{e}_{uv}^{\text{conv}} + \mathbf{t}_{\tau_{uv}}. \quad (3.8)$$

3.3.3 Binary Edge Head

A linear layer projects each $\tilde{\mathbf{e}}_{uv}$ to 2 logits corresponding to the binary label of whether the directed edge $u \rightarrow v$ exists in the ground-truth DAG. This **edge head** is used only during training as an auxiliary supervision signal; its predictions are discarded at inference. Its role is discussed in detail in Section 3.7.

3.4 Structural Attention Bias

3.4.1 Motivation

Standard multi-head self-attention assigns attention weights α_{ij} between all pairs of edge representations $(\tilde{\mathbf{e}}_i, \tilde{\mathbf{e}}_j)$ purely from learned query-key dot products, with no consideration of their topological relationship in the causal graph. This ignores a strong prior: edges that share an endpoint – or form a directed chain through a node of interest – are structurally more relevant to each other than unrelated background edges. For example, in the chain $X \rightarrow K \rightarrow Y$, the edges $X \rightarrow K$ and $K \rightarrow Y$ should attend to each other more strongly than to a background edge $A \rightarrow B$.

3.4.2 Six Topological Relationship Types

For each ordered pair of edges (i, j) in the graph, a topological relationship type $\tau(i, j) \in \{0, \dots, 5\}$ is assigned based on shared endpoints. Given edges $e_i = (u_1, v_1)$ and $e_j = (u_2, v_2)$, the six types are defined in Table 3.1.

Table 3.1: Six topological relationship types between edge pairs used in the structural attention bias.

Type	Condition	Causal interpretation
Reverse	$u_1 = v_2, v_1 = u_2$	Same variable pair, opposite direction
Shared source	$u_1 = u_2, v_1 \neq v_2$	Common cause / fork
Shared target	$v_1 = v_2, u_1 \neq u_2$	Common effect / collider
Forward chain	$v_1 = u_2$	Mediator chain $u_1 \rightarrow v_1 \rightarrow v_2$
Backward chain	$v_2 = u_1$	Mediator chain $u_2 \rightarrow v_2 \rightarrow v_1$
Unrelated	otherwise	No shared endpoint

3.4.3 Learned Scalar Bias per Topological Type

The attention logit between edges i and j is augmented with a learnable scalar bias specific to their topological relationship:

$$A_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j + b_{\tau(i,j)}}{\sqrt{d}}, \quad (3.9)$$

where $\mathbf{q}_i, \mathbf{k}_j \in \mathbb{R}^d$ are the standard query and key projections, d is the head dimension, and $b_{\tau} \in \mathbb{R}$ is a single learnable scalar for each of the six types. This introduces only 6 additional parameters per attention head while encoding explicit causal structural prior knowledge. All bias scalars are initialized to zero ($b_{\tau} = 0$ for all τ), so training begins from standard unbiased self-attention, preserving learned representations from early epochs while gradually allowing the model to adjust attention weights according to causal topology. Two layers of multi-head self-attention with structural bias are applied sequentially to aggregate information across the full edge set.

3.4.4 Why This Succeeds Where Other Architectural Modifications Failed

Several other architectural extensions – parallel observation cross-attention branches (v3, v4), observation-level transformers, node-centric aggregation (v9) – all produce results at or below baseline. The distinguishing factor is that structural attention bias adds *a structured inductive prior with a minimal parameter budget* (6 scalars per head), rather than additional unconstrained capacity. The bias does not learn a new representation from scratch – it only learns the *relative priority* among topologically distinct edge pairs. This interpretation is validated by the experimental result: structural bias improves Balanced Accuracy by +2.65% over the same input channels without it (v8b: 76.94% vs. v11: 79.59%), a gain larger than any other single architectural change in the thesis.

3.5 Node Visual Representation

3.5.1 Motivation

The 1D edge tensor representation encodes each variable pair (u, v) as a functional curve – sorted observations and regression residuals indexed along the ordering of u . This representation is powerful for capturing edge-level directional asymmetry but is inherently *pairwise*: it describes the marginal relationship between u and v rather than the joint visual pattern of a node K with respect to both X and Y simultaneously. The proposed **node visual representation** complements the 1D pipeline by rendering the joint scatter distribution of K against X and Y as a two-dimensional density image. This captures visual spatial structure – clustering, curvature, heteroscedasticity, and residual patterns – that is invisible in any single 1D marginal curve. For instance, a Confounder ($K \rightarrow X, K \rightarrow Y$) produces a characteristic scatter pattern in the (K, X) and (K, Y) planes that is geometrically distinguishable from a Collider ($X \rightarrow K, Y \rightarrow K$).

3.5.2 8-Channel Scatter Density Image

For each non- X/Y node K , an 8-channel 32×32 image $\mathbf{I}_K \in \mathbb{R}^{8 \times 32 \times 32}$ is constructed from the $n = 1000$ observations. The 8 channels are grouped into two sets with separate smoothing parameters.

Channels 1–4: Raw scatter density ($\sigma_{\text{raw}} = 4.0$). Each channel renders the scatter plot of two variables as a 2D histogram, Gaussian-smoothed to produce a continuous density:

$$\mathbf{I}_K^{(1)} = \rho(K, X), \quad \mathbf{I}_K^{(2)} = \rho(K, Y), \quad \mathbf{I}_K^{(3)} = \rho(X, K), \quad \mathbf{I}_K^{(4)} = \rho(Y, K), \quad (3.10)$$

where $\rho(a, b)$ denotes the smoothed 32×32 joint density image of variables a (horizontal axis) and b (vertical axis), with each axis min-max normalized to $[-1, 1]$. The four views provide the full bidirectional visual context of K with respect to X and Y . A larger smoothing $\sigma_{\text{raw}} = 4.0$ is used to smooth out observation noise while preserving the coarse spatial structure of the joint distribution.

Channels 5–8: ANM residual scatter density ($\sigma_{\text{ANM}} = 2.0$). To encode directional asymmetry in the 2D representation, channels 5–8 replace one variable in each pair with the ANM residual computed at bandwidth $h = 0.5$:

$$\mathbf{I}_K^{(5)} = \rho(K, r_{X \rightarrow K}), \quad \mathbf{I}_K^{(6)} = \rho(X, r_{K \rightarrow X}), \quad \mathbf{I}_K^{(7)} = \rho(K, r_{Y \rightarrow K}), \quad \mathbf{I}_K^{(8)} = \rho(Y, r_{K \rightarrow Y}), \quad (3.11)$$

where $r_{u \rightarrow v}$ denotes the ANM residual of v regressed on u . If the true causal direction is $u \rightarrow v$, the residual $r_{u \rightarrow v}$ is approximately independent of u , producing a uniform horizontal scatter in $\rho(u, r_{u \rightarrow v})$; the reverse-direction residual retains visible nonlinear structure. A finer smoothing $\sigma_{\text{ANM}} = 2.0$ is used for these channels to preserve the fine-grained directional asymmetry patterns that coarser smoothing would obscure.

Why 8 channels, not 12. An earlier variant added 4 channels encoding multivariate kernel regression coefficient maps, scoring 80.42% versus 80.47% for the 8-channel version. The kernel coefficient represents a *slope* – a derived linear relationship – while the image axes represent *observed variable values*. The spatial inductive bias of Conv2D is appropriate for density patterns but inappropriate for coefficient maps, which have no meaningful spatial structure along the observation axis. Removing the 4 coefficient channels recovers and marginally improves performance.

3.5.3 Hierarchical Conv2D Feature Extractor

The 8-channel 32×32 image $\mathbf{I}_K$ is processed by a hierarchical Conv2D network that progressively downsamples the spatial resolution:

$$32 \times 32 \xrightarrow{\text{Conv2D}} 16 \times 16 \xrightarrow{\text{Conv2D}} 8 \times 8 \xrightarrow{\text{Conv2D}} 4 \times 4 \xrightarrow{\text{pool}} \mathbf{e}_K^{2D} \in \mathbb{R}^{64}. \quad (3.12)$$

Each Conv2D block uses Group Normalization and GELU activations consistent with the 1D pipeline. The final AdaptiveAvgPool collapses the 4×4 spatial map to a $d = 64$ dimensional node visual embedding $\mathbf{e}_K^{2D}$.

3.6 Fusion and Node Classification

3.6.1 Combining Edge Context with Visual Representation

For each node K , the final classification embedding is formed by combining five sources of information: the four directional edge context embeddings from the Edge Context Encoder and the visual embedding from the Node Visual Classifier. Let $\mathbf{e}_{K \rightarrow X}, \mathbf{e}_{K \rightarrow Y}, \mathbf{e}_{X \rightarrow K}, \mathbf{e}_{Y \rightarrow K} \in \mathbb{R}^{64}$ be the four attended edge embeddings, and let $\mathbf{e}_K^{2D} \in \mathbb{R}^{64}$ be the node’s visual embedding. These are combined via a learned linear merge operator:

$$\mathbf{h}_K = \text{MergeOperator}(\mathbf{e}_{K \rightarrow X}, \mathbf{e}_{K \rightarrow Y}, \mathbf{e}_{X \rightarrow K}, \mathbf{e}_{Y \rightarrow K}, \mathbf{e}_K^{2D}) \in \mathbb{R}^{64}. \quad (3.13)$$

The node head then maps $\mathbf{h}_K$ to 8-class logits:

$$\hat{\mathbf{y}}_K = \text{Linear}_{64 \rightarrow 8}(\mathbf{h}_K). \quad (3.14)$$

3.6.2 Necessity of Edge Context

The edge context is not merely supplementary – it is the essential component that provides graph-wide structural information to the node classifier. When the entire 1D edge pipeline is removed and nodes are classified from their 2D visual embeddings alone (v26c), Balanced Accuracy collapses to **51.51%** compared to **80.47%** for the full model. A variant using mean pooling of all node embeddings as a substitute for edge context (v28b) achieves only 46.39%. This confirms that the 2D visual representation captures local distributional structure of K relative to X and Y , but cannot reason about K ’s causal role within the larger graph – a task that requires the global relational information encoded by the attended edge embeddings. The binary edge loss, by supervising the edge pipeline with ground-truth adjacency matrix entries, ensures these embeddings remain causally meaningful throughout training.

3.7 Loss Function and Training Objective

3.7.1 Dual-Head Loss with Lambda Weighting

The model is trained with two loss terms. The **primary node loss** $\mathcal{L}_{\text{node}}$ is a weighted cross-entropy over the 8 causal role classes, with class weights set inversely proportional to class frequency in the training set:

$$\mathcal{L}_{\text{node}} = - \sum_{c=1}^8 w_c \cdot \mathbf{1}[y = c] \cdot \log \hat{p}_c, \quad w_c \propto \frac{1}{n_c}, \quad (3.15)$$

where n_c is the number of training samples of class c , y is the ground-truth label, and $\hat{p}_c$ is the predicted probability for class c . The **auxiliary edge loss** $\mathcal{L}_{\text{edge}}$ is a binary weighted cross-entropy over all $p(p-1)$ directed edges, predicting whether each edge exists in the ground-truth DAG. The total training loss is:

$$\mathcal{L} = \lambda \cdot \mathcal{L}_{\text{edge}} + (1 - \lambda) \cdot \mathcal{L}_{\text{node}}, \quad \lambda = 0.7. \quad (3.16)$$

3.7.2 Rationale for $\lambda = 0.7$

The choice of $\lambda = 0.7$ assigns the majority of the training gradient to the auxiliary edge loss. This is counterintuitive – the edge head is discarded at inference and the primary objective is node classification – but it is empirically necessary. The 2D node classifier receives its structural context entirely from the attended edge embeddings (Equation 3.13). If λ is small (node loss dominates early training), the edge pipeline receives insufficient gradient and converges to representations optimized for the node head’s immediate needs rather than for causally meaningful edge structure, producing degenerate context that degrades the node merger quality. Increasing λ to 0.7 ensures the edge embeddings are well-supervised throughout training. This mechanism is confirmed by ablation: a configuration favoring node loss ($\lambda = 0.3$) achieves 78.77%, while $\lambda = 0.7$ achieves 80.47% – a difference of +1.70% with identical input channels and architecture, differing only in the loss weighting.

3.8 X/Y Remap Data Augmentation

3.8.1 Strategy

The training set contains 23,500 dataset instances, each with a designated pair (X, Y) satisfying $X \rightarrow Y$ in the ground-truth DAG. The 8-class labels for all remaining nodes K are defined relative to this specific (X, Y) pair. However, a dataset instance with p variables contains multiple valid directed edges in the ground-truth DAG, each of which could serve as a treatment-outcome pair. The X/Y remap augmentation exploits this: for each directed edge $A \rightarrow B$ in the ground-truth DAG, a new training sample is generated by treating (A, B) as the new (X', Y') and recomputing 8-class labels for all remaining nodes K' according to their causal relationship with A and B in the same DAG. This produces approximately $11.2\times$ more training samples without collecting any additional data, yielding a training set of approximately 263,000 samples. The augmentation is justified by the symmetry of the classification problem: the 8 causal roles are defined purely by the local structure of the graph around the designated (X, Y) pair, and this local structure occurs with equal validity for any directed edge in the DAG.

3.8.2 Efficient Implementation

Naively recomputing the 8-channel edge tensors for each remap would multiply the preprocessing cost by $\sim 11\times$. The key efficiency insight is that the kernel regression computation (channels 3–8) depends only on the observation matrix $\mathbf{D}$, not on the choice of (X, Y) . Therefore, `coeff_map` and `resid_map` are computed *once* per graph. For each remap, only three operations are required: (1) edge type re-assignment (cheap name-substitution); (2) node image reconstruction from cached coefficient and residual maps (cheap histogram operations); and (3) label re-derivation from the adjacency matrix (a lookup table query). No kernel regression is recomputed. The augmented dataset is stored as 6 shard files, each containing $\sim 50,000$ samples, to avoid loading the full 263K dataset into RAM simultaneously. A custom `ShardGroupedSampler` ensures that each training batch draws samples from the same shard, minimizing disk I/O during training.

3.9 Training Configuration

All models are trained with the following fixed configuration: AdamW optimizer [17] with a cosine annealing learning rate schedule [18], initial learning rate 10^{-3} , batch size 64, 10 training epochs on the augmented 263K dataset (equivalent to ~ 110 epochs over the base 23,500 samples), and 32-bit floating-point precision throughout. A Gaussian noise augmentation of standard deviation 0.01 is applied to edge tensors during training to improve generalization. Training uses Distributed Data Parallel (DDP) across two NVIDIA RTX 5880 GPUs. The model has approximately 280K parameters in total (edge encoder: ~ 188 K; node Conv2D extractor: ~ 92 K; node merge and head: negligible).

Chapter 3 Summary

This chapter presented the five components of the proposed method. The **8-channel edge tensor** (Section 3.2) encodes each variable pair as multi-bandwidth functional curves and ANM residuals, preserving the full shape of conditional relationships. The **Edge Context Encoder** (Section 3.3) processes these tensors via shared Conv1D and produces a global graph context through structural self-attention. The **Structural Attention Bias** (Section 3.4) injects topology-aware causal priors into the attention mechanism with only 6 additional learnable scalars per head. The **Node Visual Representation** (Section 3.5) constructs 8-channel scatter density images per node and extracts complementary 2D visual features via Conv2D. The **Fusion** step (Section 3.6) combines edge context with visual features for the final 8-class prediction, with the dual-head loss (Section 3.7) ensuring that the edge context encoder remains causally well-supervised throughout training. Finally, **X/Y remap augmentation** (Section 3.8) multiplies the training set by $\sim 11\times$ without additional data collection. The combined system achieves **81.082%** Balanced Accuracy on the CrunchDAO private test set.

Chapter 4

Evaluation Methodology

This chapter describes the dataset, evaluation metric, experimental baselines, and experiment design used to assess the proposed method.

4.1 Dataset

4.1.1 Data Source

All experiments use the official dataset from the ADIA Lab Causal Discovery Challenge [1, 4]. According to the post-competition report by the organizers [6], the full competition dataset contains approximately 47,000 synthetic graph instances split across training and test sets. Through the CrunchDAO platform, this thesis accesses: a **training set** of 23,500 dataset instances with ground-truth adjacency matrices; a **public test set** of 1,880 dataset instances, scored by the CrunchDAO platform during the competition; and a **private test set**, a held-out partition not revealed until the final submission, on which the final model achieves **81.082%** Balanced Accuracy – the headline result of this thesis. Neither the training nor the test sets provide direct class labels; instead, each instance is accompanied by an adjacency matrix $\mathbf{A} \in \{0, 1\}^{p \times p}$ representing the ground-truth causal DAG, from which class labels are derived as described below.

4.1.2 Label Derivation from Adjacency Matrix

The 8-class label for each variable $K \notin \{X, Y\}$ is derived from $\mathbf{A}$ by examining four binary bits: $\mathbf{A}_{K,X}$ ($K \rightarrow X$), $\mathbf{A}_{X,K}$ ($X \rightarrow K$), $\mathbf{A}_{K,Y}$ ($K \rightarrow Y$), $\mathbf{A}_{Y,K}$ ($Y \rightarrow K$). The class is determined by the rule table in Table 4.1.

4.1.3 Dataset Characteristics

Each dataset instance contains $n = 1000$ observations over $p \in \{3, \dots, 10\}$ variables, with X and Y pre-identified and $\mathbf{A}_{X,Y} = 1$ guaranteed. Instances with $p > 3$

Table 4.1: Label derivation rules from the adjacency matrix for variable K .

Label	$K \rightarrow X$	$X \rightarrow K$	$K \rightarrow Y$	$Y \rightarrow K$
Confounder	1	0	1	0
Mediator	0	1	1	0
Collider	0	1	0	1
Cause of X	1	0	0	0
Cause of Y	0	0	1	0
Consequence of X	0	1	0	0
Consequence of Y	0	0	0	1
Independent	0	0	0	0

contribute more than one variable K , so the total number of (K, label) training pairs exceeds 23,500. The eight classes are not uniformly distributed: Independent is the most frequent class, while Mediator and Collider have the fewest samples. This imbalance directly motivates the use of inverse-frequency class weights in the loss function (Section 3.7) and Balanced Accuracy as the evaluation metric. According to the competition report [6], the data are generated from structural causal models with nonlinear mechanisms (neural networks, Gaussian processes, Bayesian models) over six synthetic graph types and real-world food-web graphs, with all variables normalized to $[-1, 1]$.

4.2 Evaluation Metric: Balanced Accuracy

The evaluation metric for all experiments in this thesis is **Balanced Accuracy** (BA), defined as the mean per-class recall across all $C = 8$ classes:

$$\text{BA} = \frac{1}{C} \sum_{c=1}^C \frac{\#\{\text{correct predictions of class } c\}}{\#\{\text{true instances of class } c\}}. \quad (4.1)$$

Balanced Accuracy is the appropriate metric for this problem for two reasons. First, the class distribution is imbalanced: reporting overall accuracy would overweight performance on the majority class (Independent) and obscure failures on rare classes (Mediator, Collider). Second, all eight causal roles are equally important from an application perspective – misclassifying a Mediator as a Consequence

of X is as costly as misclassifying an Independent as a Confounder. An uninformed model that always predicts the most frequent class achieves approximately $\frac{1}{8} = 12.5\%$ Balanced Accuracy. The competition organizers’ supervised baseline achieved $\sim 58\%$, and the first-place solution achieved 76.70% [6].

4.3 Experimental Baselines

4.3.1 External Baselines

Competition first-place solution [7]: The reference method, achieving **76.70%** on the public leaderboard. Architecture: 3-channel edge tensor $\rightarrow$ Stem Conv1D $\rightarrow$ 5 residual Conv1D blocks $\rightarrow$ GAP $\rightarrow$ edge-type embedding $\rightarrow$ 2-layer self-attention $\rightarrow$ node head (8-class) + edge head (binary, auxiliary). All improvements in this thesis are measured relative to a faithful reimplementation of this solution (v2: **73.96%** local / **73.61%** public), confirming reproducibility.

Machine learning baselines: Two ML baselines represent the feature engineering + tree-based paradigm. v10 uses a LightGBM classifier on scalar statistical features achieving **63.11%**. v13 implements a full-stack pipeline with 300+ engineered features, outputs from four classical causal discovery algorithms (PC, LiNGAM, NOTEARS, GES), an XGBoost/LightGBM/CatBoost ensemble, and a GNN refinement stage, achieving **72.64%**. These represent the maximum reasonable effort under the ML paradigm and provide a rigorous lower bound for assessing the superiority of the deep learning approach on this task.

4.3.2 Internal Ablation Baselines

The key internal baselines for measuring component contributions are: **v2** (3ch reimplementation), measuring the starting point; **v8b** (8ch 1D, no structural bias), isolating the contribution of multi-bandwidth channels and ANM residuals; **v11** (8ch 1D + structural bias), isolating structural attention bias; **v11+xyaug**, the previous best at 81.14% local / 80.82% public, serving as the reference for measuring the v26e 2D pipeline gain; **v26c** (2D only, no edge pipeline), demonstrating the necessity of edge context; and **v26e base** (8ch 2D + edge encoder, no XY aug), isolating the 2D pipeline contribution before augmentation.

4.4 Experimental Setup

4.4.1 Hardware Environment

All experiments are conducted on a server with two NVIDIA RTX 5880 GPUs (48 GB combined VRAM), a 48-core CPU for parallelizing preprocessing, and 251 GB RAM. Training uses Distributed Data Parallel (DDP) across both GPUs for the full augmented dataset. The dual-path transformer (v4) required an A100 80 GB GPU due to out-of-memory errors with the 40 GB configuration.

4.4.2 Two-Tier Evaluation Procedure

Each model version is evaluated via: (1) **local test set** – 1,880 held-out instances from the public test split, with known ground-truth adjacency matrices, providing immediate feedback after training; (2) **CrunchDAO public leaderboard** – selected versions submitted to the platform for official scoring, denoted “LB” in result tables; and (3) **CrunchDAO private test set** – the final proposed model (v26e 8ch + XY augmentation) submitted once for scoring on the hidden private partition, achieving **81.082%**. The private test score is reported once as the final evaluation.

4.4.3 Experiment Design Principles

Experiments follow an ablation-first protocol: train and evaluate on the base 23,500-sample dataset first; if the result is positive, run with XY augmentation ($\sim 263\text{K}$ samples); submit to the public leaderboard only for results that show clear local improvement, to avoid overfitting to the public test set. This protocol ensures that every architectural or feature decision is grounded in evidence from the ablation chain before incurring the additional computational cost of the augmented training run.

4.5 Complete Experiment Summary

Table 4.2 presents all model versions with their local and public leaderboard scores, organized chronologically and grouped by experiment category. Detailed analysis

of these results is presented in Chapter 5.

Chapter 4 Summary

This chapter described the evaluation setup: 23,500 training instances and 1,880 local test instances from the ADIA Lab dataset, Balanced Accuracy as the sole evaluation metric, and a three-tier scoring procedure (local, public LB, private). The baseline landscape spans two ML approaches (v10: 63.11%, v13: 72.64%), the reimplemented DL baseline (v2: 73.96%), and the previous best (v11+xyaug: 81.14%). The proposed final model (v26e 8ch + XY augmentation) achieves **81.082%** on the private test set. Detailed analysis of each result is presented in Chapter 5.

Table 4.2: Complete experiment results. “Local” = 1,880-sample local test set; “LB” = CrunchDAO public leaderboard; “Private” = hidden test set (final submission only). $\Delta v2$ = change relative to reimplementation baseline. Dashes indicate not submitted or scored.

Version	Description	Local	LB	$\Delta v2$
<i>ML baselines</i>				
v10	LightGBM on scalar features	63.11%	—	−10.85
v13	ML fullstack (300+ feat, 4 algos)	72.64%	—	−1.32
<i>DL reimplementation & early exploration</i>				
v2	3ch baseline (reimplementation)	73.96%	73.61%	—
v3	+5ch (partial corr, resid asym)	72.46%	72.13%	−1.50
v4	Dual-path transformer	72.77%	68.76%	−1.19
v5	+scalar stat injection	73.57%	72.84%	−0.39
v6	New attention variant	72.28%	—	−1.68
<i>Feature engineering improvements</i>				
v5m	+multi-bw kernel (5ch)	75.44%	74.80%	+1.48
v5m+xyaug	+XY augmentation	77.91%	77.55%	+3.95
v8b	8ch (5 kernel + 3 ANM)	76.94%	—	+2.98
v8b+xyaug	+XY augmentation	80.26%	80.22%	+6.30
<i>Architectural exploration (negative results)</i>				
v9	Node-centric (reduce $O(p^2)$)	75.43%	—	+1.47
v9b	Dual-path node-centric	75.65%	—	+1.69
v14	Node-rich scalar features	76.00%	—	+2.04
v15	Multi-tower (1D + scalar)	78.66%	—	+4.70
v17	XY-aware TTA	76.51%	—	+2.55
<i>Structural attention bias</i>				
v11	v8b + structural bias	79.59%	—	+5.63
v11+xyaug	+XY augmentation	81.14%	80.82%	+7.18
v11 3-seed ensemble	ensemble	80.92%	80.27%	+6.96
<i>2D scatter density pipeline</i>				
v25b	4ch scatter density (node)	78.99%	—	+5.03
v25a	4ch smoothed heatmap	79.56%	—	+5.60
v26b	8ch pairwise conv2d	79.17%	—	+5.21
v26c	8ch node 2D only (no edge)	51.51%	—	−22.45
v26d	edge loss favored ($\lambda = 1.0$)	80.37%	—	+6.41
v26d node-fav	node loss favored ($\lambda = 0.3$)	78.77%	—	+4.81
v26e 12ch	12ch (incl. coeff channels)	80.42%	—	+6.46
v26e 8ch	8ch (remove coeff channels)	80.47%	—	+6.51
v26f	learnable smoothing	79.37%	—	+5.41
v28b	node deep sets, no edge	46.39%	—	−27.57

Chapter 5

Results and Analysis

This chapter presents and analyzes the complete experimental results. Results are grouped by theme: Section 5.1 addresses the DL versus ML comparison; Section 5.2 analyzes the feature engineering contributions; Section 5.3 covers the structural attention bias; Section 5.4 analyzes the 2D scatter density pipeline and its ablations; and Section 5.6 synthesizes the overall lessons learned.

5.1 Deep Learning vs. Machine Learning: A Fair Comparison

The thesis includes two machine learning baselines: v10 (LightGBM on scalar features, 63.11%) and v13 (full-stack pipeline, 72.64%). These results are substantially below the proposed final model (81.082% private). This section analyzes the gap with mechanistic precision, acknowledging both the limitation specific to this task and the genuine advantages of the ML paradigm.

5.1.1 v10: LightGBM Baseline

Version v10 extracts scalar features – pairwise correlations, ANM scores, partial correlations – and trains a LightGBM classifier, achieving 63.11%. The result is low because the feature set encodes conditional dependence *strength* but not the *shape* of the conditional expectation function, which is the key discriminating signal for classes like Mediator versus Consequence of X: both have $X \rightarrow K$ in their DAG structure, but they differ in whether $K \rightarrow Y$ exists, a difference that is reflected in the curvature and tail behavior of the regression curve $\hat{m}_{X \rightarrow K}$ rather than in any scalar summary of it.

5.1.2 v13: Full ML Fullstack

Version v13 represents the maximum reasonable engineering effort under the ML paradigm: 300+ features including Pearson, Spearman, and partial correlations;

conditional mutual information; HSIC; ANM scores; BIC scores from PC, LiNGAM, NOTEARS, and GES algorithms; an XGBoost/LightGBM/CatBoost ensemble; a GNN refinement stage; and stacking. This achieves 72.64% – marginally below the DL baseline reimplementation (v2: 73.96%) and -8.5% below the final proposed model.

The gap is mechanistic, not incidental. Every scalar feature in v13 is a fixed aggregation of the $n = 1000$ observations: a single number derived from the joint distribution. The information loss is irreversible – once compressed to a scalar, the spatial structure of the joint density and the shape of the conditional expectation curve cannot be recovered. In contrast, the 8-channel edge tensor (channels 2–8) retains the full length- n curve, and Conv1D learns which aspects of that curve discriminate causal roles. The functional form of the relationship between u and v – its curvature, heteroscedasticity, and asymmetry – is precisely the information that distinguishes the eight roles, and it is the information that scalar features irreversibly discard.

This is not a general criticism of the ML paradigm. ML approaches have genuine advantages: interpretability (each feature has a direct statistical meaning), no GPU requirement, and faster development cycles. For this specific task, however, the discriminating signal lives in the shape of nonlinear functional relationships rather than in aggregate statistics, making the information loss from scalar compression decisive.

Table 5.1: Summary comparison of deep learning and machine learning approaches.

Approach	Version	Local BA	Advantages	Limitation on this task
ML (minimal)	v10	63.11%	Fast, interpretable	Scalar features only
ML (fullstack)	v13	72.64%	No GPU, interpretable	Irreversible compression
DL (proposed)	v26e+aug	81.19%	Learns from full curve	GPU required

5.2 Feature Engineering Contributions

5.2.1 Multi-Bandwidth Kernel Regression (+1.48%)

Adding two extra kernel regression channels at $h \in \{0.2, 1.0\}$ alongside the original $h = 0.5$ (v5m: 75.44% vs. v2: 73.96%) yields a consistent +1.48% improvement. Each bandwidth captures a different smoothness scale of the nonlinear conditional expectation: $h = 0.2$ is sensitive to sharp local transitions, $h = 1.0$ reflects global monotone trends. The gain is consistent across the public leaderboard (+1.97% on LB), confirming it is not an artefact of the local test split.

5.2.2 ANM Residual Channels (+1.50% over v5m)

Adding three ANM residual channels to produce an 8-channel tensor (v8b: 76.94%) improves over the 5-channel version (v5m: 75.44%). The improvement is concentrated in Collider (+6.4%) and Consequence of X (+1.5%), both of which require detecting *directionality* – whether the edge runs toward or away from the node – that is encoded in the residual asymmetry between $r_{u \rightarrow v}$ and $r_{v \rightarrow u}$.

5.2.3 XY Augmentation (+3.32% on v8b, +1.55% on v11)

XY remap augmentation delivers the largest single improvement across all versions. On v8b, the gain is +3.32% (v8b+xyaug: 80.26%); on v11, it is +1.55% (v11+xyaug: 81.14%). The smaller gain on v11 reflects partial information overlap with the structural attention bias, which already encodes topological X/Y relationships in the architecture; augmentation contributes less new perspective when the architecture already sees multiple (X', Y') viewpoints implicitly. The augmentation is consistent across the public leaderboard: v8b+xyaug achieves 80.22% LB, and v11+xyaug achieves 80.82% LB, with a local-to-LB gap of $\leq 0.35\%$ confirming that augmentation does not induce overfitting.

5.3 Structural Attention Bias

5.3.1 Result and Mechanism (+2.65%)

Adding structural attention bias to v8b (v11: 79.59% vs. v8b: 76.94%) yields +2.65%, the largest gain from any single architectural modification in the thesis. The gain is distributed across classes, with the largest improvements in Confounder (+4.46%), Collider (+3.40%), and Mediator (+1.04%) – the three classes whose definitions involve triangle motifs (chain, fork, collider) that are directly encoded in the six topological relationship types.

5.3.2 Contrast with Failed Architectural Experiments

Versions v3, v4, v6, v9, and v9b all attempt architectural additions and produce results at or below baseline. The distinguishing factor between v11 and these failed experiments is not parameter count but the alignment of inductive bias with domain knowledge: the six topological relationship types encode prior knowledge about causal motifs (chain, fork, collider) that the model should attend to. Six additional scalars with this alignment (+2.65%) outperform thousands of additional parameters without it (−1.19% to −1.90%). This validates the principle that for this problem, *structured inductive bias* consistently outperforms *increased unconstrained capacity*.

5.4 2D Scatter Density Pipeline

5.4.1 Edge Context Necessity

Before analyzing the 2D pipeline’s gains, it is essential to establish that the edge context encoder is indispensable for the 2D classifier. Table 5.2 presents the ablation evidence.

The collapse to 51.51% (v26c) and 46.39% (v28b) confirms that the 2D density image captures only the *local* distributional structure of K relative to X and Y , and cannot reason about K ’s causal role without knowing the broader graph context. The edge encoder – supervised by the auxiliary binary edge loss – provides exactly this context through the four edge embeddings ($K \rightarrow X$, $K \rightarrow Y$, $X \rightarrow K$, $Y \rightarrow$

Table 5.2: Ablation: necessity of edge context for node classification. All variants use the same 8-channel node images; only the source of graph context differs.

Version	Node Context Source	Local BA
v26c	None (pure conv2d, no edge pipeline)	51.51%
v28b	Mean pooling of node embeddings	46.39%
v26e 8ch (full)	Edge encoder + structural self-attention	80.47%

K). Without it, performance is worse than the ML baseline, confirming that the “auxiliary” edge head is a structural necessity, not a regularization convenience.

5.4.2 1D vs. 2D Representation

Comparing v11 base (79.59%) and v26e base (80.47%) isolates the contribution of the 2D node image pipeline over the pure 1D edge-only approach with identical auxiliary components. Table 5.3 shows the per-class breakdown.

Table 5.3: Per-class recall: v11 (1D edge pipeline only) vs. v26e 8ch (1D + 2D fusion), base dataset without XY augmentation.

Class	v11 (1D only)	v26e 8ch (1D+2D)
Confounder	75.81%	78.59%
Collider	77.80%	75.78%
Mediator	67.29%	69.57%
Cause of X	82.91%	82.40%
Cause of Y	83.54%	84.71%
Consequence of X	79.69%	80.20%
Consequence of Y	81.96%	84.14%
Independent	87.67%	88.38%
Balanced Accuracy	79.59%	80.47%

The 2D pipeline gains most on Confounder (+2.78%), Mediator (+2.28%), and Consequence of Y (+2.18%) – classes whose distinguishing feature is the *joint* distributional pattern of K with respect to both X and Y simultaneously. The 1D edge tensor captures each pair separately; the 2D image captures the joint view.

Collider drops slightly (-2.02%), suggesting that the 1D ANM residual channels are more powerful than 2D density for detecting the collider signature (induced dependence under conditioning). This reinforces the complementary nature of the two representations: neither dominates the other across all classes.

5.4.3 Channel Count: 12ch vs. 8ch

An early 12-channel variant (v26e 12ch: 80.42%) added 4 kernel regression coefficient channels alongside the 8 density channels, scoring marginally below the 8-channel version (80.47%). The kernel coefficient encodes a per-observation slope $\partial\hat{m}(u)/\partial u$; its axis corresponds to the spatial ordering in the image, not the observed variable values. The Conv2D spatial inductive bias – appropriate for density patterns – is inappropriate for coefficient maps, causing a slight degradation. Removing the 4 coefficient channels recovers the performance, establishing the 8-channel configuration as the correct design.

5.4.4 Loss Lambda Tuning

Table 5.4 shows the effect of the edge loss weight λ on v26e 8ch (no XY augmentation).

Table 5.4: Effect of loss weight λ on v26e 8ch. $\mathcal{L} = \lambda \cdot \mathcal{L}_{\text{edge}} + (1 - \lambda) \cdot \mathcal{L}_{\text{node}}$.

Configuration	λ	Local BA
v26d node-favored	0.3	78.77%
v26d edge=node	1.0	80.37%
v26e (proposed)	0.7	80.47%

The $\lambda = 0.7$ configuration outperforms both $\lambda = 0.3$ (node-favored: -1.70%) and $\lambda = 1.0$ (equal: -0.10%). The -1.70% gap between $\lambda = 0.3$ and $\lambda = 0.7$ is a training dynamics effect: when the node loss gradient dominates early, the edge pipeline receives insufficient supervision and produces inconsistent edge embeddings that degrade the node merger quality. The $\lambda = 1.0$ vs. $\lambda = 0.7$ gap is small (-0.10%), suggesting that any configuration with sufficiently high edge weight is adequate; $\lambda = 0.7$ is chosen as it empirically performs best.

5.4.5 Final Model: v26e 8ch + XY Augmentation

The final proposed model achieves **81.19%** Balanced Accuracy on the local test set, **80.96%** on the CrunchDAO public leaderboard, and **81.082%** on the CrunchDAO private test set. Per-class results on the local test set are shown in Table 5.5.

Table 5.5: Per-class recall of the final proposed model (v26e 8ch + XY aug) on the local test set.

Class	Recall	Count (n)
Confounder	74.93%	682
Collider	78.70%	446
Mediator	73.71%	483
Cause of X	83.32%	983
Cause of Y	85.45%	2145
Consequence of X	81.29%	1571
Consequence of Y	83.72%	682
Independent	88.38%	4494
Balanced Accuracy	81.19%	

The three hardest classes – Confounder (74.93%), Collider (78.70%), and Mediator (73.71%) – remain below the overall average, consistent with competition-wide findings [6]. The improvement over the reimplement baseline (v2: 73.96%) is largest for Mediator (+13.7%), Collider (+12.7%), and Consequence of X (+8.6%).

5.5 Full Ablation Table

Table 5.6 summarizes the cumulative contribution of each proposed component as incremental gains over the preceding configuration.

Table 5.6: Incremental ablation: contribution of each component, where each row adds one component to the previous configuration.

Configuration	Local BA	Δ prev.	Component added
v2: 3ch reimplementation	73.96%	—	—
v5m: +multi-bw kernel (5ch)	75.44%	+1.48	Channels $h = 0.2$, $h = 1.0$
v8b: +ANM residuals (8ch)	76.94%	+1.50	Channels 6–8
v11: +structural attn bias	79.59%	+2.65	6-scalar topology bias
v11+aug: +XY augmentation	81.14%	+1.55	$\sim 11\times$ data
v26e 8ch: +2D node pipeline	80.47%*	—	8ch density + Conv2D
v26e+aug: +XY augmentation	81.19%	+0.72	XY aug on 2D model

*v26e base without XY aug; +0.88% over v11 base (1D-only).

5.6 Synthesis and Lessons Learned

5.6.1 Shape Information Cannot Be Recovered After Scalar Compression

The v13 ML fullstack experiment establishes this as the central lesson of the thesis. 300+ features, four causal discovery algorithms, and three gradient boosting models together achieve 72.64% – lower than every Conv1D-based model that uses even a minimal set of functional curve channels. The functional form of the relationship between u and v , captured in channels 2–8 of the edge tensor, contains causal role information that is not accessible through any scalar summary. This finding has direct implications for the design of future causal discovery systems: when the task requires distinguishing roles that differ in the shape of conditional relationships, scalar feature engineering is insufficient regardless of sophistication.

5.6.2 Structured Inductive Bias Outperforms Unconstrained Capacity

The structural attention bias (v11, +2.65% over v8b) is the only architectural modification among all 30+ experiments that consistently improves performance. Every other architectural change – parallel observation branches, dual-path transformers, node-centric attention, deep sets – produces results at or below baseline. The dis-

tinguishing factor is not parameter count but the alignment of inductive bias with domain knowledge: the six topological relationship types encode prior knowledge about causal motifs (chain, fork, collider) that the model should attend to. Six additional scalars with this alignment outperform thousands of additional parameters without it.

5.6.3 1D Curves and 2D Density Images Are Complementary

The 1D edge tensor captures directional pairwise relationships between variable pairs; the 2D density image captures joint distributional patterns of a single node relative to both X and Y simultaneously. Neither representation fully subsumes the other: the 1D pipeline is stronger for Collider (directional residual asymmetry), while the 2D pipeline is stronger for Confounder and Mediator (joint distributional structure). The fusion of both representations (v26e: 80.47%) outperforms either alone (v11: 79.59%; v26c: 51.51%).

5.6.4 Edge Context Is the Foundation, Not an Auxiliary

Despite the “auxiliary” label attached to the binary edge head, the edge pipeline is the structural foundation of the entire model. The 29-point collapse of v26c (51.51%) when the edge pipeline is removed demonstrates that no amount of visual information in the 2D density images can compensate for the absence of graph-wide context. The binary edge loss – supervised by ground-truth adjacency matrices – forces the edge embeddings to encode causally meaningful structure that the node merger depends on. This is essential supervised context encoding, not auxiliary regularization.

Chapter 5 Summary

This chapter presented and analyzed the complete experimental results. The key findings are: (1) the ML fullstack (v13: 72.64%) confirms that scalar feature compression is irreversible and structurally insufficient for this task; (2) multi-bandwidth channels and ANM residuals each provide consistent $\approx +1.5\%$ improvements; (3) structural attention bias provides $+2.65\%$ as the only successful architectural modification, validated by contrast with failed unconstrained architectural additions;

(4) the 2D scatter density pipeline provides +0.88% over the 1D-only model with complementary per-class improvements; (5) edge context is essential – removing it collapses performance by 29 points; and (6) XY augmentation provides +1.55%–+3.32% across configurations. The final proposed model (v26e 8ch + XY augmentation) achieves **81.082%** Balanced Accuracy on the private test set, surpassing the competition first-place solution (76.70%) by +4.38% and the reimplement baseline (73.96%) by +7.12%.

Conclusion

Tóm tắt bài toán, giải pháp và các kết quả đạt được

Bibliography

- [1] CrunchDAO and ADIA Lab. Adia lab causal discovery challenge: Discovering causal relationships from observational data. <https://hub.crunchdao.com/competitions/causality-discovery>, 2024.
- [2] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2 edition, 2009.
- [3] Uzma Hasan, Emam Hossain, and Md Osman Gani. A survey on causal discovery methods for i.i.d. and time series data, 2023.
- [4] CrunchDAO. Adia lab causal discovery (competition documentation). <https://docs.crunchdao.com/competitions/competitions/adia-lab-causal-discovery>, 2024.
- [5] Peter Spirtes, Clark Glymour, and Richard Scheines. *Causation, Prediction, and Search*. MIT Press, 2 edition, 2000.
- [6] Emanuele Olivetti, Vincent Zoonekynd, Patrick Yam, Marcos López de Prado, Guido W. Imbens, and Miguel A. Hernán. Can AI learn causal structure?: Evidence from ADIA lab’s causal discovery challenge. SSRN Working Paper, 2025. Available at SSRN: <https://ssrn.com/abstract=6125566>.
- [7] thetourney. ADIA lab causal discovery challenge – 1st place solution. <https://thetourney.github.io/adia-report/>, 2024.
- [8] Dan Geiger, Tom S. Verma, and Judea Pearl. d-separation: From theorems to algorithms, 2013.
- [9] CrunchDAO. Causality discovery (competition hub). <https://hub.crunchdao.com/competitions/causality-discovery>, 2024.
- [10] Elizbar A. Nadaraya. On estimating regression. *Theory of Probability & Its Applications*, 9(1):141–142, 1964.
- [11] Geoffrey S. Watson. Smooth regression analysis. *Sankhyā: The Indian Journal of Statistics, Series A*, 26(4):359–372, 1964.

- [12] Shohei Shimizu, Patrik O. Hoyer, Aapo Hyvarinen, and Antti Kerminen. A linear non-gaussian acyclic model for causal discovery. *Journal of Machine Learning Research*, 7:2003–2030, 2006.
- [13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [14] Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.
- [15] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs), 2016.
- [16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [17] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.
- [18] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts, 2017.
- [19] Patrik O. Hoyer, Dominik Janzing, Joris M. Mooij, Jonas Peters, and Bernhard Schölkopf. Nonlinear causal discovery with additive noise models. In *Advances in Neural Information Processing Systems*, volume 21, 2009.
- [20] Alex KC. 4th place solution — ADIA lab × Crunch-DAO. <https://medium.com/@alexkiechlecornish/4th-place-solution-100k-causal-discovery-challenge-adia-lab-x-crunchdao-2024>.
- [21] Hoang Thien Nu. The 5th place solution to the ADIA causal discovery challenge 2024. <https://medium.com/@htnu/the-5th-place-solution-to-the-adia-causal-discovery-challenge-2024-236d5> 2024.